S.I.G.S.M

Programación Full Stack

Empre S.A.S.

Rol	Apellido	Nombre	C.I	Email
Coordinador	Cabrera	Martin	4919741-2	empre.sa.utu.isbo@gmail.com
Subcoordinador	Sena	Angel	5341149-0	jeansangel2003@gmail.com

Docente: Acosta Alvez, Fabian

Fecha de culminación

09/11/2026

TERCERA ENTREGA

1. Introducción..... 6

1.1. Presentación del documento..... 6

1.2. Objetivos de la tercera entrega..... 7

1.3. Estado actual del desarrollo..... 8

2. Modelo de datos definitivo..... 10

2.1. Introducción al modelo de datos..... 10

2.2. Evolución del modelo respecto a entregas anteriores..... 11

2.3. Cambios y decisiones de diseño..... 12

2.3.1. Gestión de funcionarios y roles..... 13

2.3.2. Gestión documental..... 13

2.3.3. Gestión de traslados y recursos..... 14

2.3.4. Historial de estados..... 15

2.3.5. Incorporación del módulo de encuestas..... 15

2.4. Diagrama Entidad-Relación definitivo (DER)..... 16

2.5. Descripción de las entidades y sus relaciones..... 18

2.5.1. Autenticación y funcionarios..... 18

2.5.2. Gestión documental..... 18

2.5.3. Gestión de traslados..... 19

2.5.4. Gestión de encuestas..... 20

2.6. Restricciones de integridad y reglas de negocio..... 21

2.6.1. Integridad referencial..... 21

2.6.2. Restricciones de unicidad..... 22

2.6.3. Validaciones de traslados..... 22

2.6.4. Validaciones de encuestas..... 23

2.6.5. Conservación de información e historial..... 24

3. Arquitectura del sistema..... 24
3.1. Descripción general de la arquitectura..... 24
3.2. Arquitectura cliente-servidor..... 25
3.3. Organización del frontend..... 27
3.3.1. Componentes y páginas..... 27
3.3.2. Comunicación con la API..... 28
3.3.3. Navegación y control de acceso..... 29
3.3.4. Componentes y funcionalidades compartidas..... 29
3.4. Organización del backend y la API REST..... 30
3.4.1. Endpoints..... 30
3.4.2. Handlers y servicios..... 31
3.4.3. Middleware de autenticación y autorización..... 32
3.4.4. Configuración y utilidades..... 33
3.5. Persistencia y acceso a los datos..... 33
3.5.1. Organización de la base de datos..... 34
3.5.2. Almacenamiento de archivos..... 35
3.6. Arquitectura de despliegue mediante Docker..... 35
3.6.1. Servicios principales..... 36
3.6.2. Servicios auxiliares..... 37
3.6.3. Redes y persistencia..... 37
3.7. Comunicación entre los componentes..... 38
3.7.1. Comunicación durante el desarrollo..... 38
3.7.2. Comunicación en Docker..... 39
3.7.3. Intercambio de información y manejo de respuestas..... 39
4. Diagramas de clases..... 40

4.1. Introducción y objetivo....40
4.2. Diagrama de clases del sistema....41
4.3. Descripción de las clases principales y sus relaciones....43
5. Diagramas de secuencia....44
5.1. Introducción y objetivo....44
5.2. Diagrama de autenticación de funcionarios....45
5.3. Diagrama de gestión documental....47
5.4. Diagrama de solicitud y gestión de traslados....50
5.4.1. Registro de una solicitud de traslado....50
5.4.2. Actualización del estado de un traslado....52
5.5. Diagrama de gestión de encuestas....55
5.5.1. Creación de una encuesta....55
5.5.2. Registro de respuestas de una encuesta....57
6. Testing funcional....59
6.1. Introducción y objetivos de las pruebas....59
6.2. Metodología y entorno de pruebas....60
6.3. Pruebas de autenticación y autorización....61
6.4. Pruebas del módulo de documentación....62
6.5. Pruebas del módulo de encuestas....62
6.6. Pruebas del módulo de traslados....63
6.7. Pruebas de validación y manejo de errores....64
6.7.1. Control de acceso a funcionalidades restringidas....65
6.7.2. Asignación de recursos a un traslado....65
6.7.3. Actualización de estados de traslado....66
6.8. Resultados generales de las pruebas....67

7. Conclusiones..... 69

7.1. Resultados del desarrollo..... 69

7.2. Limitaciones conocidas..... 71

7.3. Posibles mejoras futuras..... 72

8. Bibliografía y referencias..... 73

9. Anexos..... 75

9.1. Repositorio GitHub del proyecto..... 75

9.2. Scripts SQL definitivos..... 76

9.3. Documentación técnica complementaria..... 76

1. Introducción

El presente documento corresponde a la tercera y última entrega de la asignatura Programación Full Stack, en el marco del Proyecto de Egreso 2026 del Bachillerato Tecnológico en Tecnologías de la Información del Instituto Superior Brazo Oriental (I.S.B.O.).

El proyecto, denominado S.I.G.S.M. (Sistema Integral de Gestión de Servicios Médicos), consiste en el desarrollo de una aplicación web orientada a la digitalización y optimización de distintos procesos administrativos del Hospital de Clínicas de Montevideo. Su desarrollo está a cargo de Empre S.A.S., empresa conformada en el ámbito académico para llevar adelante el proyecto.

Esta entrega tiene como finalidad presentar la estructura definitiva del sistema, documentar las decisiones técnicas adoptadas durante su desarrollo y demostrar el funcionamiento de las principales funcionalidades implementadas mediante diagramas y pruebas funcionales.

A diferencia de las entregas anteriores, centradas principalmente en la planificación, el diseño inicial y los primeros avances de implementación, esta instancia se enfoca en documentar la solución desarrollada y justificar los cambios realizados a lo largo del proyecto.

1.1. Presentación del documento

El presente documento reúne la documentación técnica correspondiente al estado final del desarrollo de S.I.G.S.M., haciendo especial énfasis en el diseño de la base de datos, la arquitectura de la aplicación, los diagramas UML y las pruebas realizadas sobre sus principales funcionalidades.

En primer lugar, se presenta el modelo de datos definitivo, identificando las modificaciones realizadas respecto a las versiones anteriores y justificando las decisiones de diseño adoptadas. Se incluye un nuevo Diagrama Entidad-Relación (DER) que representa las entidades, relaciones y restricciones que conforman la base de datos actual.

Posteriormente, se describe la arquitectura del sistema, explicando la organización de sus diferentes componentes, la comunicación entre el frontend y el backend, el acceso a la base de datos y la infraestructura utilizada para su ejecución.

A continuación, se incorporan los diagramas de clases y de secuencia correspondientes a los procesos principales de la aplicación. Estos permiten representar tanto la organización interna del software como las interacciones que se producen entre sus componentes durante la ejecución de determinadas operaciones.

Finalmente, se documentan las pruebas funcionales realizadas, con el objetivo de comprobar el comportamiento de los módulos desarrollados, sus validaciones y el tratamiento de situaciones de error.

1.2. Objetivos de la tercera entrega

El objetivo principal de esta entrega es documentar y presentar la solución técnica desarrollada para S.I.G.S.M., reflejando la evolución del proyecto desde sus primeras etapas de diseño hasta su implementación actual.

Para cumplir con este objetivo general, se establecen los siguientes objetivos específicos:

- Presentar el modelo de datos definitivo, incluyendo las entidades y relaciones incorporadas durante el desarrollo.
- Identificar y justificar las modificaciones realizadas respecto al diseño de la base de datos presentado en entregas anteriores.
- Describir la arquitectura del sistema y la organización de sus componentes.
- Documentar las principales clases e interacciones del software mediante diagramas UML.
- Representar mediante diagramas de secuencia los procesos más relevantes de la aplicación.
- Documentar las pruebas funcionales realizadas sobre los distintos módulos del sistema.
- Verificar el cumplimiento de las principales reglas de negocio y validaciones implementadas.
- Identificar las limitaciones conocidas y posibles mejoras futuras.

De esta manera, se busca proporcionar una visión técnica completa del proyecto, facilitando tanto su evaluación académica como su comprensión por parte de otros desarrolladores o responsables de su mantenimiento.

1.3. Estado actual del desarrollo

Al momento de realizar esta entrega, S.I.G.S.M. cuenta con una aplicación web que integra diferentes funcionalidades administrativas mediante una arquitectura modular.

El sistema fue desarrollado utilizando React para el frontend, junto con Material UI para la construcción de las interfaces. El backend está implementado en PHP 8.2 mediante una API REST, mientras que la persistencia de la información se realiza utilizando MariaDB.

La aplicación se encuentra organizada en diferentes módulos funcionales, entre los que se destacan:

- **Autenticación y administración de funcionarios**: permite gestionar el acceso al sistema mediante sesiones y un modelo de permisos basado en roles, incluyendo la administración de cuentas de funcionarios.
- **Gestión documental**: permite administrar categorías y documentos institucionales, incluyendo su carga, modificación, visualización, descarga y acceso público controlado.
- **Gestión de encuestas**: permite crear y administrar formularios, configurar preguntas y opciones de respuesta y recopilar la información proporcionada por los usuarios.
- **Gestión y seguimiento de traslados**: permite registrar solicitudes, asignar vehículos y funcionarios, consultar la disponibilidad de recursos y actualizar el estado de los traslados durante su ejecución.
- **Gestión de vehículos**: permite administrar los vehículos disponibles y consultar información relacionada con su tipo y estado operativo.

Durante el desarrollo también se incorporaron funcionalidades complementarias, como el historial de estados de los traslados, los filtros de búsqueda, las validaciones de datos, la gestión de permisos y la internacionalización de la interfaz.

Por otra parte, se implementó una configuración de despliegue mediante Docker Compose, que permite ejecutar el frontend, el backend y la base de datos en contenedores independientes. Esta configuración facilita la instalación del sistema en diferentes equipos y permite conservar la información mediante volúmenes persistentes.

La evolución del proyecto también implicó modificaciones en el diseño inicial de la base de datos y en la organización interna del código. Estas modificaciones

respondieron a nuevas necesidades funcionales identificadas durante la implementación y a la búsqueda de una estructura más organizada, mantenible y coherente con los requerimientos del sistema.

En esta etapa, el trabajo se centra principalmente en consolidar la documentación técnica, verificar los flujos funcionales y registrar los resultados obtenidos, así como las limitaciones que puedan permanecer.

Por último, es importante señalar que S.I.G.S.M. continúa siendo un proyecto académico. Su eventual implementación dentro de la infraestructura del Hospital de Clínicas dependerá de las autorizaciones, configuraciones e integraciones que determine el Departamento Técnico de Informática (DTI) de la institución.

2. Modelo de datos definitivo

2.1. Introducción al modelo de datos

El modelo de datos de S.I.G.S.M. fue diseñado para almacenar y relacionar la información utilizada por los distintos módulos de la aplicación, garantizando su integridad, consistencia y disponibilidad.

Para su implementación se utiliza MariaDB como sistema gestor de bases de datos relacional, empleando el motor de almacenamiento InnoDB, que permite trabajar con claves foráneas, transacciones y restricciones de integridad referencial.

Desde las primeras etapas del proyecto se adoptó un diseño modular, organizando las entidades según las funcionalidades a las que pertenecen. Esta organización se mantuvo durante el desarrollo, aunque fue necesario incorporar nuevas tablas,

atributos y restricciones para responder a los requerimientos funcionales que se fueron implementando.

El modelo definitivo está compuesto por 18 tablas, distribuidas entre los módulos de autenticación, documentación, gestión de traslados y encuestas. Si bien cada módulo administra información específica, todos comparten determinadas entidades, principalmente las relacionadas con los funcionarios del sistema.

En los siguientes apartados se describen las modificaciones más relevantes realizadas durante el desarrollo, las decisiones adoptadas y la estructura definitiva de la base de datos.

2.2. Evolución del modelo respecto a entregas anteriores

El modelo de datos presentado durante las primeras entregas permitió establecer una estructura inicial para el almacenamiento de la información. Sin embargo, a medida que avanzó la implementación de los módulos, fue necesario ampliar y ajustar determinados aspectos del diseño.

Estos cambios no implicaron una modificación completa de la arquitectura de datos, sino una evolución progresiva del modelo original, conservando sus entidades principales y adaptándolas a las funcionalidades desarrolladas.

En el módulo de autenticación se mantuvo la estructura basada en funcionarios, roles y asignaciones de roles. Este diseño permite que un mismo funcionario disponga de diferentes permisos dentro de la aplicación sin necesidad de duplicar sus datos personales.

En el módulo de documentación se conservaron las entidades principales de categorías y documentos, manteniendo las relaciones que permiten identificar al

funcionario responsable de su creación y la categoría a la que pertenece cada documento.

El módulo de traslados presentó una evolución más significativa debido a la incorporación de funcionalidades relacionadas con la asignación de recursos, la disponibilidad de vehículos y funcionarios, el seguimiento de estados y la consulta del historial de operaciones.

El diseño actual contempla distintas entidades auxiliares para representar los tipos de vehículos, elementos transportados, tipos de traslado y estados posibles. Asimismo, incorpora una tabla destinada a registrar los cambios de estado, permitiendo conservar información histórica sobre el desarrollo de cada operación.

Por otra parte, se completó el diseño correspondiente al módulo de encuestas. Este incorpora entidades destinadas a administrar formularios, preguntas, opciones de respuesta y respuestas proporcionadas por los usuarios, permitiendo almacenar la información de manera estructurada.

Como resultado, el modelo actual representa de forma más precisa las necesidades de la aplicación y proporciona una estructura adecuada para las operaciones implementadas en el backend.

2.3. Cambios y decisiones de diseño

Durante el desarrollo se tomaron distintas decisiones orientadas a mejorar la organización de los datos, evitar redundancias y garantizar el cumplimiento de las reglas de negocio.

A continuación, se detallan las principales características y decisiones reflejadas en el diseño definitivo.

2.3.1. Gestión de funcionarios y roles

Se mantuvo una relación de muchos a muchos entre funcionarios y roles mediante la tabla intermedia rol_usuario.

Esta estructura permite asignar múltiples roles a un funcionario, evitando la creación de cuentas independientes para cada responsabilidad que desempeñe dentro de la institución.

Además de los roles administrativos utilizados para gestionar los módulos, el sistema contempla roles asociados a las funciones del personal, como conductor, enfermero y médico.

La separación entre funcionarios y roles permite reutilizar la información de los usuarios en diferentes partes del sistema y facilita futuras ampliaciones del modelo de permisos.

2.3.2. Gestión documental

Las entidades categoria y documento conservaron su estructura general, manteniendo la relación que permite organizar los documentos según su categoría.

Cada documento almacena sus metadatos, incluyendo título, descripción, fecha de subida, estado y ruta del archivo físico.

Se decidió mantener los archivos fuera de la base de datos, almacenándolos en el sistema de archivos del servidor. De esta manera, MariaDB conserva únicamente la información necesaria para identificarlos y administrarlos.

También se mantiene un atributo de estado activo, que permite controlar la disponibilidad de categorías y documentos sin necesidad de eliminarlos físicamente en todas las operaciones.

2.3.3. Gestión de traslados y recursos

El modelo del módulo de traslados fue ampliado para contemplar de manera más precisa las características de cada solicitud y los recursos necesarios para realizarla.

La entidad traslado concentra la información principal de cada operación, incluyendo el tipo de traslado, el elemento transportado, origen, destino, prioridad, fechas, horarios y estado actual.

También contiene referencias al vehículo, conductor y enfermero asignados, así como al funcionario solicitante y al responsable de gestionar la solicitud.

Se incorporó una diferenciación entre horarios estimados y reales, permitiendo registrar tanto la planificación como determinados momentos efectivos de la operación.

Otra decisión importante fue incorporar la entidad compatibilidad_transporte, utilizada para establecer qué tipos de vehículos pueden transportar determinados tipos de elementos.

Esta relación evita almacenar las compatibilidades directamente en cada vehículo y permite administrar las reglas de transporte mediante un catálogo independiente.

Finalmente, se incorporó un atributo de versión en los traslados, destinado a facilitar el control de modificaciones concurrentes y evitar que una actualización sobrescriba información modificada previamente por otro usuario.

2.3.4. Historial de estados

Se incorporó la entidad historial_estado_traslado para registrar los cambios de estado realizados durante el ciclo de vida de cada solicitud.

Esta tabla almacena el traslado asociado, el estado registrado, la fecha y hora del cambio, el funcionario responsable y una observación opcional.

La decisión de utilizar una entidad independiente responde a la necesidad de conservar la trazabilidad de las operaciones.

De esta manera, mientras que la tabla traslado mantiene el estado actual, el historial permite consultar los estados registrados anteriormente sin sobrescribir la información histórica.

2.3.5. Incorporación del módulo de encuestas

El módulo de encuestas incorpora cinco entidades: encuesta, pregunta, opcion_respuesta, respuesta_encuesta y detalle_respuesta.

Se decidió separar la definición de los formularios de las respuestas proporcionadas por los usuarios.

Esta estructura permite modificar y administrar las encuestas sin mezclar sus preguntas con la información recopilada durante su utilización.

Las preguntas pueden ser abiertas, cerradas o de selección, y cada una dispone de atributos que permiten determinar su obligatoriedad, estado y posición dentro del formulario.

Asimismo, las encuestas pueden configurarse como anónimas o identificadas, contemplando el almacenamiento opcional de una cédula en los registros de respuesta cuando corresponda.

La separación entre respuestas generales y detalles permite asociar múltiples respuestas a las preguntas de un mismo formulario, manteniendo la integridad de sus relaciones.

2.4. Diagrama Entidad-Relación definitivo (DER)

El Diagrama Entidad-Relación definitivo representa la estructura actual de la base de datos de S.I.G.S.M., incluyendo las entidades implementadas, sus atributos principales, claves primarias, claves foráneas y relaciones.

A diferencia de los diagramas elaborados durante las primeras etapas, esta versión se corresponde con los scripts SQL utilizados para inicializar la base de datos de la aplicación.

Para facilitar su interpretación, las entidades se encuentran organizadas visualmente según el módulo al que pertenecen, diferenciando autenticación, documentación, traslados y encuestas.

También se representan las relaciones compartidas entre módulos, especialmente aquellas que involucran a los funcionarios, quienes pueden intervenir en distintas operaciones dependiendo de sus responsabilidades.

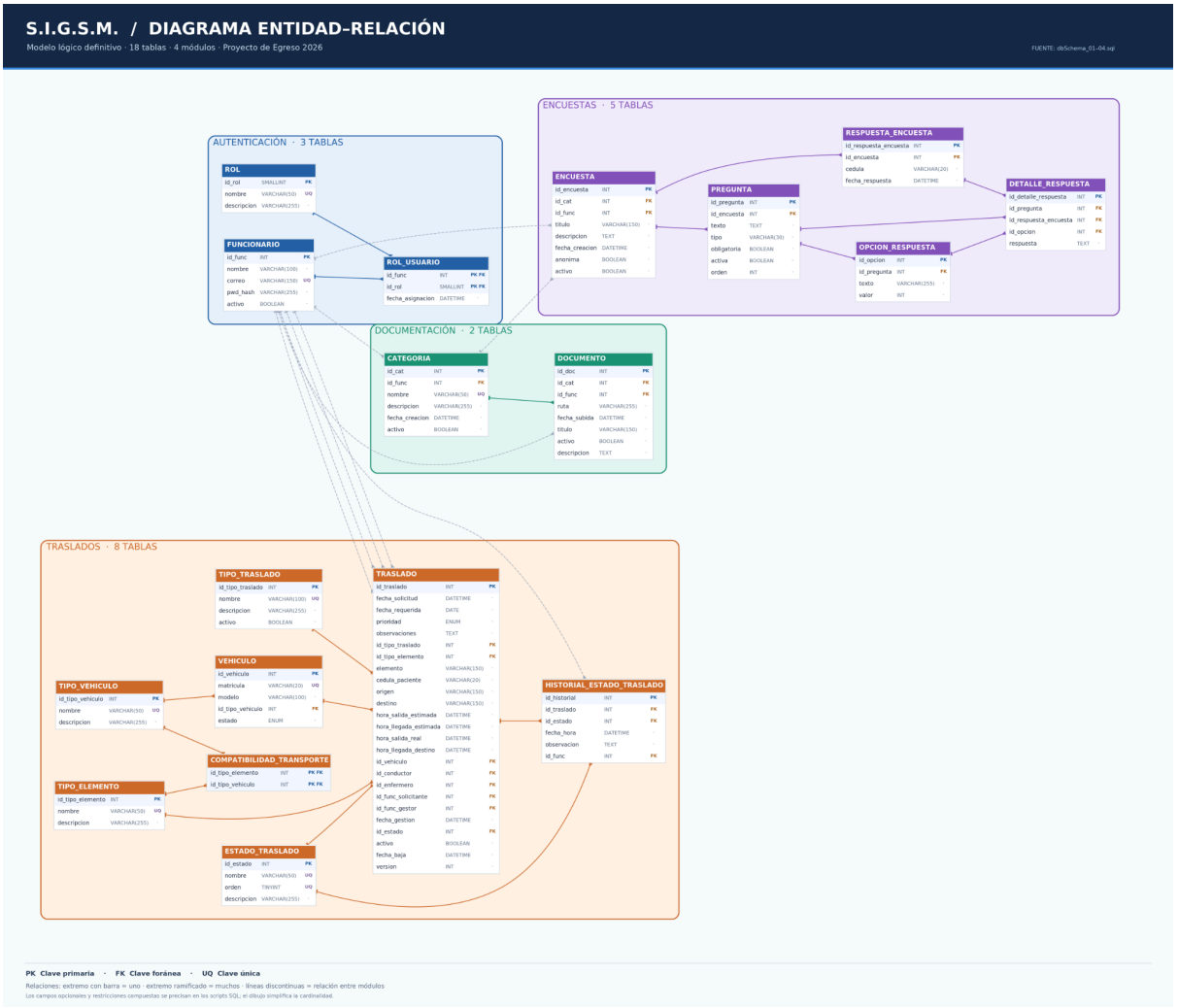


Figura 1. Diagrama Entidad-Relación definitivo de S.I.G.S.M. Elaboración propia a partir del esquema SQL del proyecto.

El diagrama permite identificar cómo se relacionan los diferentes componentes del sistema y proporciona una representación general del modelo de persistencia implementado.

La definición completa de las tablas, atributos, tipos de datos, restricciones e índices se encuentra disponible en los scripts SQL del repositorio oficial.

2.5. Descripción de las entidades y sus relaciones

La base de datos se encuentra organizada en cuatro grupos principales de entidades, correspondientes a las funcionalidades desarrolladas.

2.5.1. Autenticación y funcionarios

El módulo de autenticación está compuesto por tres entidades principales:

- **funcionario**: almacena los datos de identificación, correo electrónico, contraseña cifrada mediante hash y estado de las cuentas.
- **rol**: contiene los diferentes roles disponibles dentro del sistema y sus descripciones.
- **rol_usuario**: relaciona funcionarios y roles, permitiendo que un funcionario tenga múltiples roles asignados.

La relación entre funcionario y rol es de muchos a muchos, implementada mediante rol_usuario.

La entidad funcionario también se relaciona con otras tablas del sistema, permitiendo identificar a los responsables de determinadas operaciones administrativas.

2.5.2. Gestión documental

El módulo de documentación utiliza dos entidades:

- **categoria**: permite organizar los documentos en grupos, almacenando su nombre, descripción, estado, fecha de creación y funcionario responsable.

- **documento**: contiene los metadatos de los archivos cargados, incluyendo título, descripción, ruta, fecha de subida, estado y referencias a la categoría y al funcionario responsable.

La relación entre categorías y documentos es de uno a muchos, dado que una categoría puede contener múltiples documentos, mientras que cada documento pertenece a una única categoría.

2.5.3. Gestión de traslados

El módulo de traslados está compuesto por ocho entidades:

- **tipo_vehiculo**: catálogo de los tipos de vehículos disponibles.
- **vehiculo**: almacena la matrícula, modelo, tipo y estado operativo de cada vehículo.
- **tipo_elemento**: catálogo de los elementos que pueden ser transportados.
- **tipo_traslado**: identifica las diferentes modalidades de traslado.
- **compatibilidad_transporte**: establece las combinaciones permitidas entre tipos de elementos y tipos de vehículos.
- **estado_traslado**: contiene los posibles estados de una solicitud y su orden dentro del flujo de seguimiento.
- **traslado**: registra la información principal de las solicitudes, sus recursos asignados, horarios y estado actual.
- **historial_estado_traslado**: conserva los cambios de estado registrados durante el ciclo de vida de cada traslado.

Entre las relaciones principales se destaca la existente entre tipo_vehiculo y vehiculo, donde un tipo puede estar asociado a múltiples vehículos.

La relación entre tipo_elemento y tipo_vehiculo es de muchos a muchos, resuelta mediante compatibilidad_transporte.

Cada traslado se encuentra asociado a un tipo de traslado, un tipo de elemento, un estado actual y un funcionario solicitante. Además, puede disponer de un vehículo, conductor, enfermero y gestor asignados.

Por último, cada traslado puede tener múltiples registros en historial_estado_traslado, permitiendo conservar los diferentes cambios de estado realizados.

2.5.4. Gestión de encuestas

El módulo de encuestas incorpora cinco entidades:

- **encuesta**: almacena la información general del formulario, como título, descripción, fecha de creación, estado, anonimato y funcionario responsable.
- **pregunta**: contiene las preguntas pertenecientes a una encuesta, incluyendo tipo, obligatoriedad, estado y orden.
- **opcion_respuesta**: almacena las opciones disponibles para determinadas preguntas.
- **respuesta_encuesta**: representa cada formulario respondido, incluyendo su fecha y una identificación opcional.
- **detalle_respuesta**: registra las respuestas individuales asociadas a las preguntas de un formulario.

Una encuesta puede contener múltiples preguntas y recibir múltiples formularios respondidos.

Cada pregunta puede disponer de diferentes opciones de respuesta, mientras que cada registro de respuesta_encuesta puede contener múltiples detalles asociados a las preguntas contestadas.

Esta organización permite separar la estructura de los formularios de los datos recopilados, evitando redundancias y facilitando el tratamiento posterior de las respuestas.

2.6. Restricciones de integridad y reglas de negocio

Además de definir las relaciones entre entidades, el modelo incorpora restricciones destinadas a garantizar la consistencia de la información almacenada.

Estas restricciones se implementan mediante claves primarias, claves foráneas, índices únicos y condiciones de validación definidas en SQL.

En determinadas operaciones, también se complementan con validaciones realizadas desde el backend, especialmente cuando las reglas dependen del estado del sistema o de información relacionada con varias entidades.

2.6.1. Integridad referencial

Las relaciones entre las diferentes tablas se encuentran definidas mediante claves foráneas.

Esto permite garantizar que los registros asociados a otras entidades hagan referencia a información existente.

Por ejemplo, no es posible registrar un documento asociado a una categoría inexistente ni asignar a un traslado un vehículo que no se encuentre registrado en la base de datos.

Las acciones de actualización y eliminación de claves relacionadas se encuentran controladas mediante las opciones CASCADE y RESTRICT, según las características de cada relación.

2.6.2. Restricciones de unicidad

Se establecieron restricciones UNIQUE sobre aquellos atributos que deben mantener valores únicos dentro del sistema.

Entre los principales ejemplos se encuentran los correos electrónicos de los funcionarios, los nombres de los roles y categorías y las matrículas de los vehículos.

En el módulo de encuestas, también se controla que dos preguntas de una misma encuesta no compartan la misma posición y que las opciones de una pregunta no repitan su valor identificador.

Estas restricciones permiten evitar duplicaciones que podrían afectar al comportamiento de la aplicación.

2.6.3. Validaciones de traslados

La entidad traslado incorpora restricciones para controlar determinados aspectos de las solicitudes.

Entre ellas se encuentran la obligatoriedad de definir un origen y un destino válidos, la identificación del paciente o elemento transportado y la coherencia entre los horarios registrados.

Los horarios estimados deben introducirse de manera conjunta y la llegada estimada debe ser posterior a la salida estimada.

También se controla que los recursos obligatorios y los datos de gestión se registren de manera coherente durante el proceso de asignación.

La base de datos verifica, además, que una llegada real al destino no sea anterior a la salida real registrada.

Las reglas más específicas, como la disponibilidad de recursos, la compatibilidad del transporte y las transiciones permitidas entre estados, se complementan mediante la lógica del backend.

2.6.4. Validaciones de encuestas

El módulo de encuestas incorpora restricciones para asegurar la validez de las preguntas y respuestas almacenadas.

Se controla que los títulos y textos obligatorios no se encuentren vacíos, que los tipos de pregunta correspondan a valores admitidos y que las posiciones de las preguntas sean válidas.

También se impide que una misma respuesta de encuesta contenga dos registros correspondientes a la misma pregunta.

Asimismo, se utiliza una clave foránea compuesta para garantizar que una opción seleccionada pertenezca efectivamente a la pregunta que se está respondiendo.

Estas restricciones permiten reducir inconsistencias y mantener la relación correcta entre formularios, preguntas, opciones y respuestas.

2.6.5. Conservación de información e historial

El modelo contempla mecanismos para preservar información relevante durante el funcionamiento de la aplicación.

En documentación y encuestas se utilizan atributos de estado que permiten controlar la disponibilidad de los registros.

En el caso de los traslados, se incorporaron atributos relacionados con la baja lógica y una entidad independiente destinada a conservar el historial de estados.

Esto permite mantener información sobre operaciones anteriores y facilita la consulta y el seguimiento administrativo de los traslados.

En conjunto, las restricciones incorporadas contribuyen a mantener la integridad de la base de datos y complementan las validaciones implementadas desde la API REST de S.I.G.S.M.

3. Arquitectura del sistema

3.1. Descripción general de la arquitectura

S.I.G.S.M. fue desarrollado siguiendo una arquitectura web modular, basada en la separación de responsabilidades entre la interfaz de usuario, la lógica de negocio y la persistencia de datos.

Esta organización permite que cada componente del sistema cumpla una función específica, facilitando el desarrollo, mantenimiento y ampliación de la aplicación.

La solución está compuesta por tres componentes principales:

- **Frontend**: desarrollado con React y Material UI, responsable de presentar las interfaces y permitir la interacción de los usuarios con las funcionalidades del sistema.
- **Backend**: implementado mediante una API REST desarrollada en PHP 8.2, encargada de procesar las solicitudes, validar la información, controlar los permisos y ejecutar las operaciones correspondientes.
- **Base de datos**: implementada en MariaDB, responsable de almacenar y administrar la información utilizada por los diferentes módulos.

La comunicación entre el frontend y el backend se realiza mediante peticiones HTTP, utilizando principalmente JSON como formato de intercambio de información. Por su parte, el backend se comunica con MariaDB mediante PDO (PHP Data Objects).

Una de las principales decisiones arquitectónicas fue mantener una separación entre estos componentes, evitando que la interfaz de usuario acceda directamente a la base de datos o concentre las reglas de negocio.

Asimismo, se adoptó una organización modular del código, agrupando los componentes y funcionalidades según su responsabilidad. Esto permite incorporar nuevas funcionalidades sin necesidad de modificar completamente la estructura existente.

3.2. Arquitectura cliente-servidor

La aplicación utiliza una arquitectura cliente-servidor, en la que los usuarios acceden al sistema mediante un navegador web y realizan operaciones que son procesadas por el servidor.

El navegador ejecuta la interfaz desarrollada en React, mientras que el servidor se encarga de proporcionar la API, ejecutar las reglas de negocio y administrar la información persistente.

El funcionamiento general de una operación es el siguiente:

1. El usuario interactúa con una funcionalidad de la interfaz.

2. React procesa la interacción y realiza una solicitud HTTP al endpoint correspondiente.

3. El backend recibe la petición y comprueba las condiciones necesarias para ejecutarla, incluyendo autenticación, autorización y validación de datos cuando corresponda.

4. Se ejecuta la lógica de negocio y, si es necesario, se realizan consultas u operaciones sobre MariaDB.

5. El backend genera una respuesta HTTP, normalmente con información en formato JSON.

6. El frontend interpreta la respuesta, actualiza la interfaz y comunica el resultado al usuario.

Este funcionamiento se aplica a las principales operaciones del sistema, como la creación de documentos, la administración de funcionarios, el registro de solicitudes de traslado y la gestión de encuestas.

La arquitectura también contempla determinadas funcionalidades de acceso público, principalmente relacionadas con la consulta de documentación y encuestas habilitadas. Estas operaciones utilizan endpoints específicos que no requieren una sesión administrativa.

La separación entre cliente y servidor permite que las reglas de negocio y las operaciones sensibles se ejecuten desde el backend, independientemente de las validaciones o restricciones visuales implementadas en React.

3.3. Organización del frontend

El frontend de S.I.G.S.M. se encuentra desarrollado utilizando React 19, junto con Vite 8 como herramienta de desarrollo y compilación.

Para la construcción de las interfaces se utiliza Material UI, que proporciona componentes reutilizables como tablas, formularios, diálogos, botones y elementos de navegación.

La aplicación emplea componentes funcionales y hooks de React para administrar estados, efectos y comportamientos de la interfaz.

El código fuente se encuentra dentro del directorio prototipos/frontend/src, organizado en diferentes carpetas según su responsabilidad.

3.3.1. Componentes y páginas

La carpeta components contiene los componentes reutilizables utilizados por las diferentes funcionalidades del sistema.

Estos se encuentran agrupados por módulos, incluyendo gestión documental, encuestas, traslados, vehículos, funcionarios y solicitudes de acceso.

Por otra parte, la carpeta pages contiene las páginas principales de la aplicación, encargadas de organizar y presentar los componentes correspondientes a cada sección.

Esta separación permite reutilizar elementos visuales y mantener una estructura organizada, evitando concentrar toda la lógica de una página en un único archivo.

3.3.2. Comunicación con la API

Las funciones encargadas de realizar solicitudes HTTP se encuentran organizadas en la carpeta apiCalls.

Estas funciones están agrupadas según el módulo al que pertenecen, permitiendo centralizar las operaciones de comunicación con el backend.

Por ejemplo, las solicitudes relacionadas con documentación, encuestas y traslados se administran desde archivos específicos, que posteriormente son utilizados por los componentes correspondientes.

Esta organización permite separar la lógica de comunicación HTTP de la representación visual de la información.

Asimismo, se utilizan funciones compartidas para procesar respuestas, gestionar errores y mantener un comportamiento consistente entre las diferentes operaciones.

3.3.3. Navegación y control de acceso

La navegación se implementa mediante React Router, que permite organizar las distintas páginas de la aplicación utilizando rutas.

Se diferencian las rutas públicas, como el inicio de sesión o la visualización de documentación habilitada, de aquellas que requieren que el funcionario se encuentre autenticado.

El frontend incorpora componentes de protección de rutas, como ProtectedRoute y RoleRoute, que permiten controlar la navegación según el estado de autenticación y los roles del usuario.

También se utiliza un archivo de configuración de permisos para determinar qué funcionalidades se encuentran disponibles para cada rol.

Es importante destacar que estas restricciones se utilizan para organizar la experiencia del usuario y evitar el acceso visual a secciones no autorizadas. La comprobación efectiva de permisos debe realizarse desde el backend, ya que las restricciones del frontend no constituyen por sí mismas un mecanismo de seguridad suficiente.

3.3.4. Componentes y funcionalidades compartidas

Además de los componentes específicos de cada módulo, el frontend incorpora funcionalidades reutilizables.

Entre ellas se encuentran los sistemas de notificaciones, las funciones auxiliares de procesamiento de datos, los componentes de diálogo y las herramientas utilizadas para administrar determinados estados de la interfaz.

También se implementó un sistema de internacionalización mediante i18next, que permite presentar la aplicación en español e inglés.

Las traducciones se encuentran organizadas en la carpeta locales, separadas por idioma y funcionalidad, facilitando su mantenimiento y ampliación.

3.4. Organización del backend y la API REST

El backend fue desarrollado utilizando PHP 8.2, sin incorporar un framework externo para la implementación de la API.

Se optó por una arquitectura modular, organizada mediante endpoints, handlers, servicios, middleware, archivos de configuración y utilidades.

Esta estructura permite distribuir las responsabilidades entre diferentes componentes, evitando concentrar las operaciones del sistema en archivos de gran tamaño.

El código se encuentra dentro del directorio prototipos/backend/api.

3.4.1. Endpoints

La carpeta public contiene los puntos de entrada de la API, organizados según el módulo al que pertenecen.

Entre sus directorios se encuentran los correspondientes a autenticación, funcionarios, categorías, documentos, encuestas, traslados, vehículos y solicitudes de acceso.

Cada endpoint recibe una petición HTTP, identifica la operación solicitada y ejecuta el procedimiento correspondiente.

Se utilizan métodos como GET, POST, PATCH y DELETE, según el tipo de operación que se necesita realizar.

Las respuestas se generan utilizando códigos de estado HTTP y, en la mayoría de las operaciones, estructuras JSON que permiten comunicar los resultados al frontend.

3.4.2. Handlers y servicios

Para las operaciones de mayor complejidad se incorporó una separación entre handlers y servicios.

Los handlers se encargan de coordinar determinadas solicitudes, procesando la información recibida y delegando las operaciones correspondientes.

Los servicios contienen la lógica de negocio y las consultas necesarias para ejecutar las funcionalidades del sistema.

Esta organización se utiliza especialmente en módulos como traslados y encuestas, donde existen operaciones que requieren múltiples validaciones, consultas o modificaciones de información relacionada.

Por ejemplo, la asignación de recursos a un traslado implica comprobar su disponibilidad durante un intervalo determinado, validar las condiciones de la solicitud y registrar la asignación correspondiente.

La separación de responsabilidades permite mantener los endpoints más reducidos y facilita la reutilización de funcionalidades.

No todas las operaciones requieren necesariamente atravesar ambas capas. Algunos endpoints sencillos ejecutan procedimientos más directos, manteniendo una estructura proporcional a su complejidad.

3.4.3. Middleware de autenticación y autorización

El backend incorpora mecanismos de autenticación basados en sesiones PHP.

La aplicación utiliza una cookie de sesión denominada sigsm_session, mediante la cual se identifica al funcionario autenticado.

Para controlar el acceso se implementaron dos funciones principales:

- **requireAuth()**: verifica que exista una sesión válida, comprueba que el funcionario continúe activo y recupera sus roles actuales desde la base de datos.
- **requireRole()**: utiliza la autenticación anterior y comprueba que el funcionario disponga de alguno de los roles autorizados para realizar la operación.

Estos mecanismos permiten separar las comprobaciones de acceso de la lógica específica de cada endpoint.

Cuando una operación requiere autenticación y el usuario no dispone de una sesión válida, la API devuelve una respuesta HTTP 401.

Si el funcionario está autenticado, pero no cuenta con los permisos necesarios, la comprobación de roles devuelve una respuesta HTTP 403.

De esta manera, las operaciones administrativas pueden aplicar controles de acceso desde el servidor, independientemente de las restricciones implementadas en la interfaz.

3.4.4. Configuración y utilidades

La carpeta config contiene los archivos responsables de administrar aspectos generales del backend, como la conexión a MariaDB, la carga de variables de entorno y la configuración de sesiones.

Por otra parte, la carpeta utils proporciona funciones reutilizables relacionadas con validaciones, procesamiento de peticiones, generación de respuestas JSON y administración de archivos.

Esta separación evita repetir procedimientos comunes en diferentes módulos y facilita la modificación de comportamientos compartidos.

3.5. Persistencia y acceso a los datos

La persistencia de la información se implementa mediante MariaDB, utilizando una base de datos relacional denominada sigsm.

El acceso a la base de datos se encuentra centralizado en la clase Database, ubicada en api/config/Database.php.

Esta clase utiliza PDO para establecer la conexión y ejecutar consultas SQL desde el backend.

Su implementación permite reutilizar una misma instancia de conexión durante la ejecución de una petición PHP, evitando crear conexiones adicionales de manera innecesaria dentro del mismo proceso de solicitud.

Las credenciales y los parámetros de conexión se obtienen mediante variables de entorno, evitando incorporar directamente las contraseñas en el código fuente.

Las consultas se realizan mediante PDO y se utilizan sentencias preparadas con parámetros para las operaciones que requieren incorporar valores proporcionados por la aplicación.

3.5.1. Organización de la base de datos

La base de datos se encuentra dividida lógicamente según los módulos funcionales del sistema.

Las entidades relacionadas con funcionarios y roles permiten administrar la autenticación y autorización.

Las entidades de documentación y encuestas almacenan la información necesaria para organizar contenidos, formularios y respuestas.

Por su parte, las entidades de traslados administran las solicitudes, los recursos asociados, los estados y el historial de las operaciones.

Las relaciones entre las tablas se encuentran definidas mediante claves primarias y foráneas, complementadas con restricciones destinadas a mantener la integridad de los datos.

El modelo definitivo y sus modificaciones se encuentran documentados en el apartado 2 del presente documento.

3.5.2. Almacenamiento de archivos

Además de la información almacenada en MariaDB, S.I.G.S.M. utiliza el sistema de archivos para conservar los documentos cargados por los funcionarios.

Los archivos físicos se almacenan dentro del directorio storage, mientras que la base de datos conserva los metadatos necesarios para su administración.

Esta separación permite evitar el almacenamiento de archivos binarios directamente en las tablas, manteniendo diferenciados los documentos y sus registros asociados.

Por este motivo, los procedimientos de respaldo y recuperación deben contemplar tanto la base de datos como los archivos almacenados.

3.6. Arquitectura de despliegue mediante Docker

Para facilitar la instalación y ejecución del sistema se incorporó una configuración de despliegue utilizando Docker y Docker Compose.

Esta configuración permite ejecutar los componentes de S.I.G.S.M. dentro de contenedores independientes, evitando la necesidad de instalar y configurar manualmente todas las dependencias en el sistema anfitrión.

El despliegue está definido en el archivo prototipos/docker/compose.yaml.

3.6.1. Servicios principales

La configuración utiliza tres servicios principales:

Frontend

El servicio frontend utiliza Nginx para servir los archivos estáticos generados durante la compilación de React mediante Vite.

También actúa como proxy inverso para las solicitudes dirigidas a la API.

Es el único servicio que publica un puerto hacia el equipo anfitrión, utilizando de forma predeterminada el puerto 8080.

Backend

El servicio backend utiliza una imagen basada en PHP 8.2 y Apache.

Apache recibe las solicitudes dirigidas a la API y ejecuta los archivos PHP correspondientes.

El servicio se comunica con MariaDB mediante la red interna de Docker y no necesita publicar directamente su puerto hacia el exterior.

Base de datos

El servicio db utiliza MariaDB 11.4 para almacenar la información del sistema.

Durante la inicialización de una instalación nueva se ejecutan los cuatro scripts SQL que definen la estructura de la base de datos.

El servicio incorpora una comprobación de disponibilidad que permite verificar que MariaDB se encuentre inicializada y pueda responder a las consultas esperadas.

3.6.2. Servicios auxiliares

Además de los componentes principales, la configuración incorpora dos servicios auxiliares.

El servicio admin-init se ejecuta durante la inicialización para crear el funcionario administrador inicial y asignarle los permisos correspondientes.

Por otra parte, seed-demo permite cargar datos ficticios mediante un perfil opcional de Docker Compose.

Este último servicio está orientado a instalaciones de demostración y pruebas, por lo que no forma parte de la ejecución habitual de la aplicación.

Ambos servicios realizan tareas puntuales y finalizan una vez completadas sus operaciones.

3.6.3. Redes y persistencia

Los contenedores se encuentran conectados mediante una red interna de Docker Compose, que permite la comunicación entre servicios utilizando sus nombres.

Por ejemplo, el backend utiliza el nombre db para conectarse con MariaDB, sin necesidad de conocer una dirección IP fija del contenedor.

Para conservar la información se utilizan dos volúmenes persistentes:

- **db_data**: almacena los archivos correspondientes a MariaDB.
- **docs_data**: almacena los documentos y archivos utilizados por la aplicación.

Estos volúmenes permiten mantener la información aunque los contenedores sean detenidos o reconstruidos, siempre que los volúmenes no sean eliminados.

La configuración facilita realizar actualizaciones independientes del frontend y backend, sin necesidad de modificar directamente los datos persistentes.

La instalación y administración de este entorno se encuentran explicadas en el manual de instalación y mantenimiento del proyecto.

3.7. Comunicación entre los componentes

La comunicación entre los diferentes componentes constituye uno de los aspectos principales de la arquitectura implementada.

Se utilizan mecanismos de comunicación diferentes según el entorno de ejecución y la responsabilidad de cada componente.

3.7.1. Comunicación durante el desarrollo

Durante el desarrollo, el frontend puede ejecutarse mediante Vite, mientras que el backend funciona de manera independiente en el servidor GNU/Linux utilizado para las pruebas.

Para facilitar esta comunicación, Vite incorpora una configuración de proxy que redirige las solicitudes realizadas a /api hacia el servidor PHP.

En este entorno, el proxy también elimina el prefijo /api antes de reenviar la solicitud, adaptándola a la estructura de rutas utilizada por el servidor PHP integrado.

Este mecanismo permite desarrollar y probar la interfaz sin necesidad de incorporar la dirección completa del backend en cada función de comunicación.

3.7.2. Comunicación en Docker

En el despliegue mediante Docker, Nginx actúa como punto de entrada de la aplicación.

Las solicitudes destinadas a las páginas de React se atienden directamente desde los archivos estáticos compilados.

En cambio, las peticiones cuya ruta comienza por /api/ son redirigidas por Nginx hacia el servicio backend.

Apache recibe dichas solicitudes mediante la configuración de rutas establecida para la API y ejecuta el endpoint PHP correspondiente.

El backend procesa la operación y, cuando es necesario, se comunica con MariaDB utilizando PDO.

Una vez completada la operación, la respuesta vuelve al navegador a través de Apache y Nginx.

Este mecanismo permite que el navegador acceda a la interfaz y a la API mediante un mismo origen, sin necesidad de publicar directamente los servicios internos.

3.7.3. Intercambio de información y manejo de respuestas

Las operaciones de la API utilizan principalmente JSON para intercambiar información entre el frontend y el backend.

En aquellos procedimientos que requieren cargar archivos, como la incorporación de documentos PDF, se utilizan peticiones que permiten transmitir información mediante formularios multipartes.

El backend se encarga de validar los datos recibidos y devolver una respuesta acompañada del código de estado HTTP correspondiente.

El frontend interpreta dicha respuesta y actualiza la interfaz según el resultado obtenido.

También se utilizan funciones compartidas para procesar errores y mostrar notificaciones al usuario, permitiendo mantener un comportamiento uniforme entre los distintos módulos.

En las operaciones autenticadas, el navegador envía la cookie de sesión correspondiente, permitiendo que el backend identifique al funcionario y compruebe sus permisos.

En conjunto, estos mecanismos proporcionan una comunicación organizada entre los componentes del sistema, manteniendo separadas las responsabilidades de presentación, procesamiento y almacenamiento de información.

4. Diagramas de clases

4.1. Introducción y objetivo

El diagrama de clases de S.I.G.S.M. permite representar la estructura orientada a objetos implementada en el backend de la aplicación, identificando las principales clases, sus responsabilidades y las dependencias existentes entre ellas.

Durante el desarrollo del proyecto, la organización de las clases evolucionó respecto a las primeras versiones. Inicialmente, la estructura contemplaba principalmente las clases Database y Traslado, mientras que la implementación definitiva incorporó diferentes servicios especializados para distribuir las operaciones de los módulos.

Esta reorganización permitió separar las responsabilidades relacionadas con la creación, consulta y modificación de información, facilitando el mantenimiento y la ampliación del sistema.

El diagrama presentado corresponde a la estructura implementada en PHP y complementa la descripción de la arquitectura desarrollada en el apartado anterior.

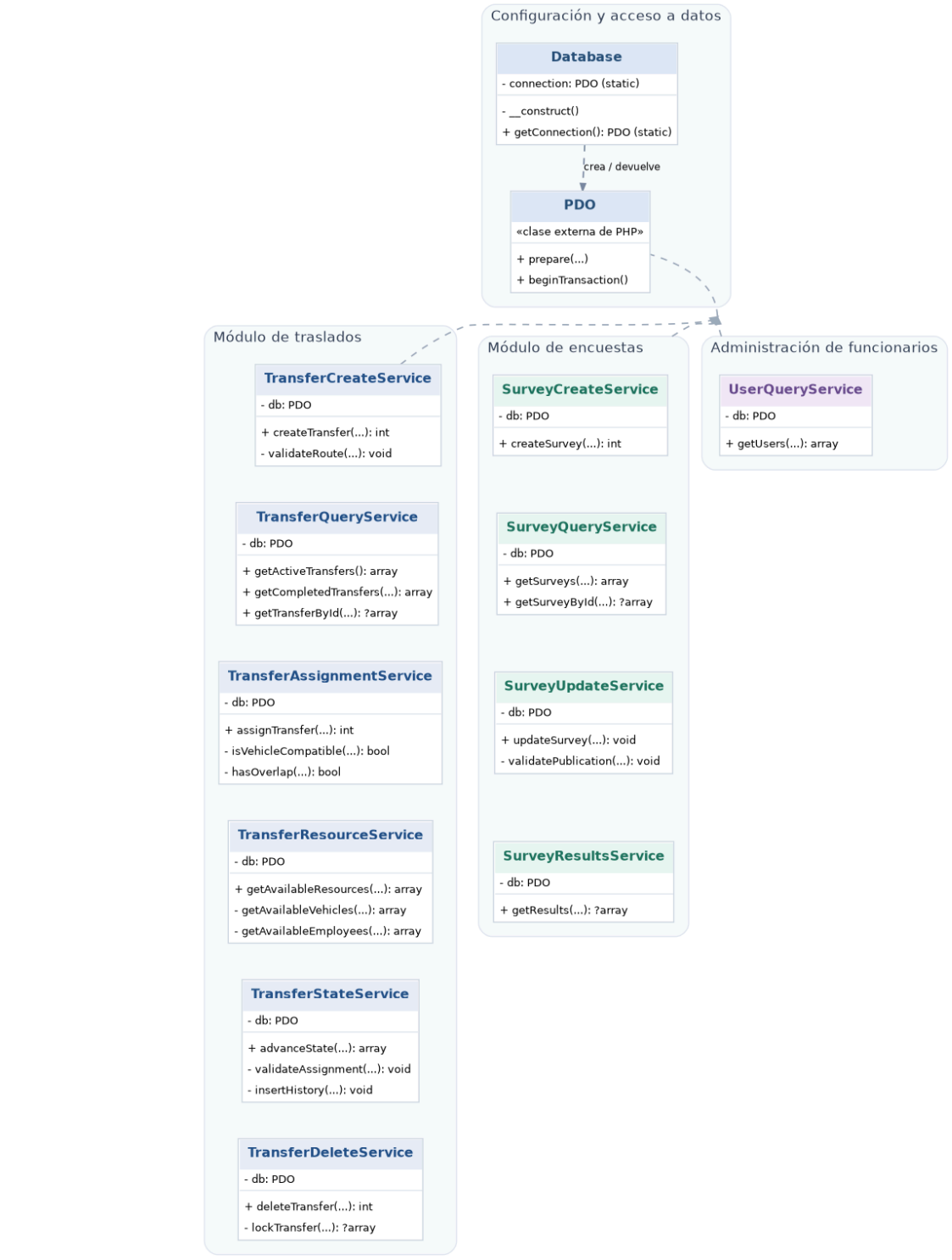
4.2. Diagrama de clases del sistema

A continuación, se presenta el diagrama de clases actualizado de S.I.G.S.M., elaborado a partir de las clases implementadas en el backend de la aplicación.

Debido a la cantidad de clases y métodos existentes, se optó por utilizar una representación simplificada, incluyendo únicamente los atributos, métodos y dependencias más representativos.

Esta decisión permite visualizar la organización general del código sin incorporar detalles de implementación que dificulten la interpretación del diagrama.

Las clases se encuentran agrupadas según su responsabilidad, diferenciando la conexión con la base de datos, la gestión de traslados, la administración de encuestas y la consulta de funcionarios.



Leyenda: + público | - privado | flecha discontinua: dependencia PDO (agrupada por módulo) | (...) parámetros abreviados

Figura 2. Diagrama de clases actualizado de S.I.G.S.M. Elaboración propia.

4.3. Descripción de las clases principales y sus relaciones

Las clases implementadas en el backend se encuentran organizadas principalmente dentro de los directorios config y services, siguiendo un diseño basado en la separación de responsabilidades.

La clase Database centraliza la conexión con MariaDB mediante PDO, proporcionando un método estático que permite obtener la conexión y reutilizarla durante la ejecución de una solicitud.

En el módulo de traslados se utilizan servicios especializados para administrar las diferentes operaciones:

- **TransferCreateService**: creación de solicitudes de traslado.
- **TransferQueryService**: consulta de solicitudes activas, finalizadas e información histórica.
- **TransferAssignmentService**: asignación de vehículos y funcionarios a los traslados.
- **TransferResourceService**: consulta de disponibilidad de recursos.
- **TransferStateService**: validación y actualización de estados.
- **TransferDeleteService**: cancelación de solicitudes permitidas.

Por otra parte, el módulo de encuestas incorpora las clases SurveyCreateService, SurveyQueryService, SurveyUpdateService y SurveyResultsService, encargadas respectivamente de la creación, consulta, modificación y obtención de resultados de las encuestas.

Finalmente, UserQueryService centraliza las operaciones de consulta de funcionarios utilizadas por las funcionalidades administrativas.

Las relaciones entre estas clases se basan principalmente en dependencias asociadas al acceso a los datos. Los servicios reciben una conexión PDO mediante sus constructores, evitando incorporar directamente la configuración de MariaDB dentro de cada clase.

No se utilizan relaciones de herencia entre los servicios, ya que cada uno mantiene responsabilidades independientes.

Asimismo, determinadas funcionalidades del sistema, como la autenticación y la gestión documental, se implementan mediante funciones y archivos PHP independientes. Por este motivo, no se incluyen como clases dentro del diagrama.

Esta organización permite mantener una estructura modular, reducir el acoplamiento entre componentes y facilitar la incorporación de nuevas funcionalidades sin modificar innecesariamente otras partes del sistema.

5. Diagramas de secuencia

5.1. Introducción y objetivo

Los diagramas de secuencia permiten representar las interacciones que se producen entre los diferentes componentes de S.I.G.S.M. durante la ejecución de sus funcionalidades principales.

A diferencia del diagrama de clases, que describe la estructura estática del sistema, los diagramas de secuencia muestran el orden temporal de las operaciones y los mensajes intercambiados entre los participantes de un proceso.

Para esta entrega se revisaron los diagramas elaborados durante las primeras etapas del proyecto, adaptándolos a la arquitectura y las funcionalidades actualmente implementadas.

Los diagramas representan la interacción entre los usuarios, la interfaz desarrollada en React, los endpoints de la API REST, los componentes internos del backend y la base de datos MariaDB.

Se seleccionaron procesos representativos de los diferentes módulos del sistema, incluyendo la autenticación de funcionarios, la gestión documental, el registro y seguimiento de traslados y la administración de encuestas.

El objetivo es proporcionar una representación gráfica del funcionamiento de estas operaciones, facilitando la comprensión de las responsabilidades de cada componente y de las validaciones que intervienen en su ejecución.

5.2. Diagrama de autenticación de funcionarios

El siguiente diagrama representa el proceso de inicio de sesión de un funcionario en S.I.G.S.M.

El proceso comienza cuando el usuario introduce su correo electrónico y contraseña desde el formulario de autenticación de la aplicación.

El frontend envía las credenciales al endpoint correspondiente de la API mediante una petición HTTP POST. El backend valida los datos recibidos y consulta la información del funcionario en MariaDB.

Si el usuario existe, se encuentra activo y la contraseña coincide con el hash almacenado, el sistema recupera sus roles asignados y genera la sesión correspondiente mediante PHP.

Posteriormente, la API devuelve una respuesta satisfactoria con la información necesaria para que el frontend actualice el estado de autenticación y permita acceder a las funcionalidades autorizadas.

El diagrama también contempla los escenarios alternativos, como credenciales incorrectas, usuarios inactivos o solicitudes con información inválida. En estos casos, el backend rechaza la autenticación y devuelve el código de estado HTTP correspondiente.

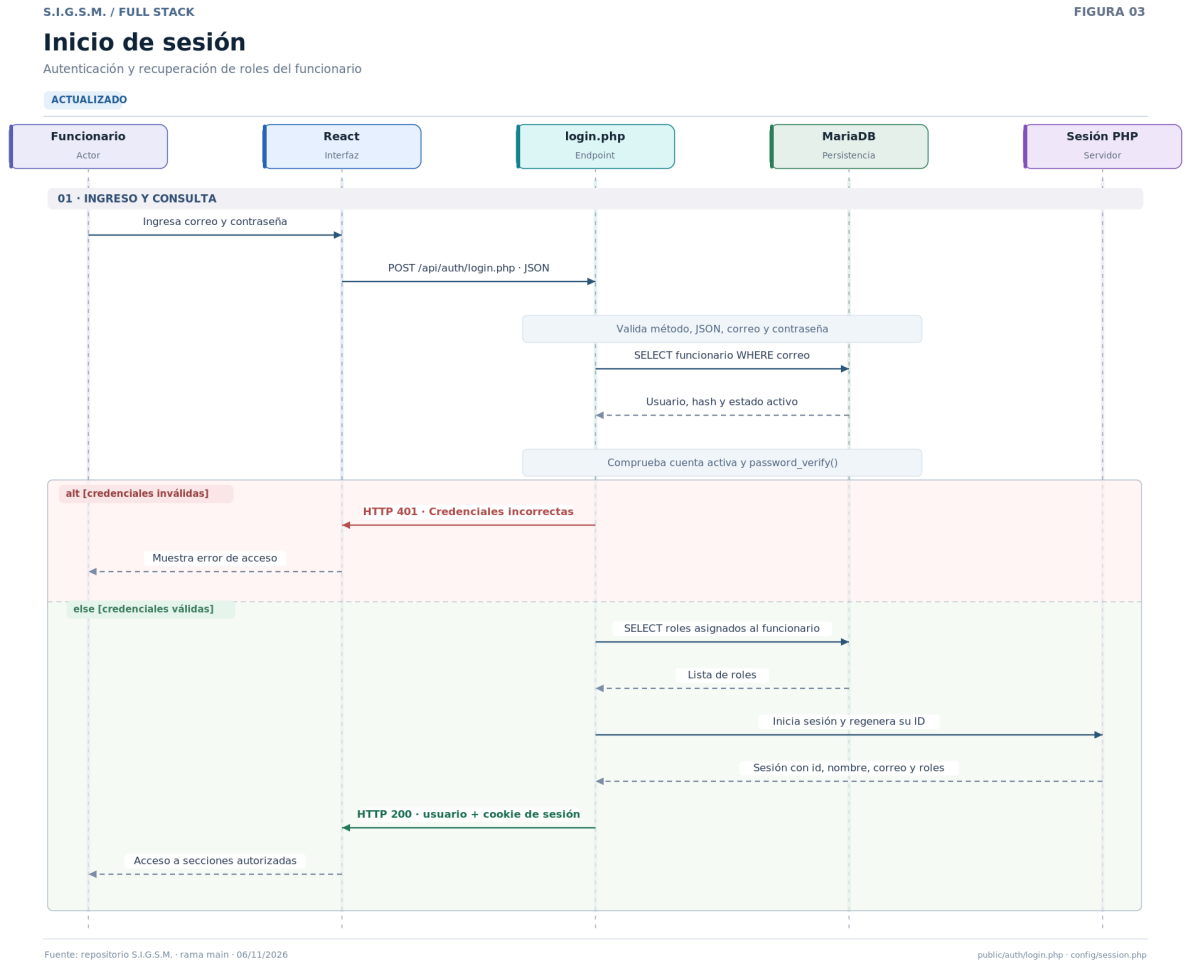


Figura 3. Diagrama de secuencia del proceso de autenticación de funcionarios.

5.3. Diagrama de gestión documental

El siguiente diagrama representa el procedimiento de incorporación de un nuevo documento al sistema.

El proceso comienza cuando un funcionario con los permisos correspondientes accede al módulo de documentación, selecciona una categoría y completa el formulario con el título, la descripción y el archivo PDF que desea almacenar.

Una vez enviada la información, el frontend realiza una petición a la API utilizando un formulario multipart, permitiendo transmitir tanto los metadatos como el archivo.

El backend comprueba la autenticación y los permisos correspondientes, valida los datos recibidos y verifica que el archivo tenga un formato admitido.

Posteriormente, consulta la existencia de la categoría seleccionada y genera un nombre único para almacenar el documento en el sistema de archivos del servidor.

Una vez guardado el archivo, se registra en MariaDB la información correspondiente, incluyendo su título, descripción, categoría, funcionario responsable y ruta de almacenamiento.

Si la operación finaliza correctamente, la API devuelve una respuesta satisfactoria y el frontend actualiza la información presentada al usuario.

El proceso también contempla situaciones de error, como datos incompletos, categorías inexistentes o problemas durante el almacenamiento. Asimismo, si el archivo fue guardado pero falla el registro en la base de datos, el backend contempla su eliminación para evitar la existencia de archivos huérfanos.

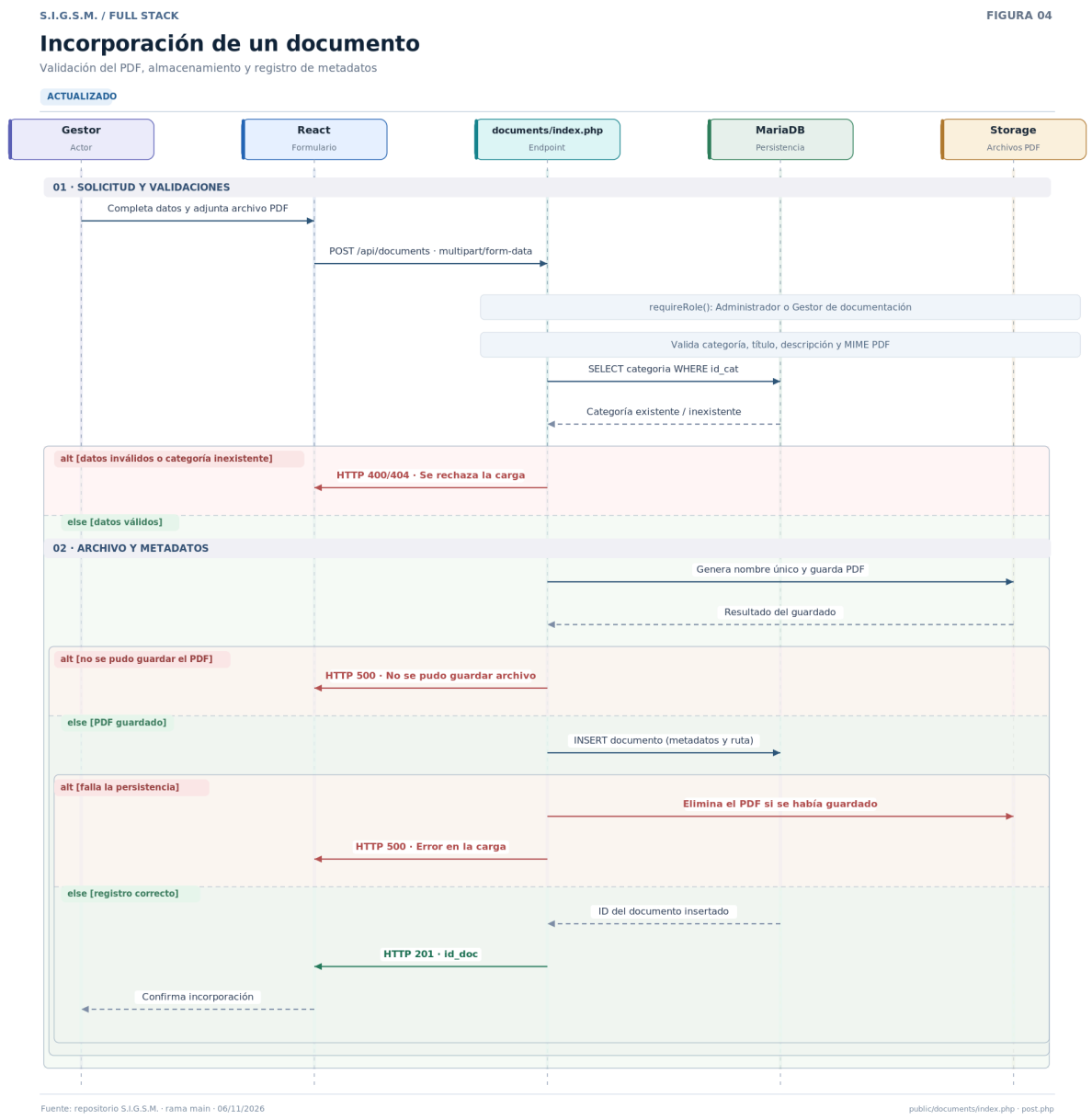


Figura 4. Diagrama de secuencia del proceso de incorporación de documentos.

5.4. Diagrama de solicitud y gestión de traslados

El módulo de traslados presenta una mayor complejidad debido a las diferentes operaciones y reglas de negocio que intervienen durante el ciclo de vida de una solicitud.

Por este motivo, se utilizan diagramas independientes para representar el registro inicial y la actualización de estados, permitiendo comprender las interacciones involucradas en cada procedimiento.

5.4.1. Registro de una solicitud de traslado

El siguiente diagrama representa el proceso de registro de una nueva solicitud de traslado.

La operación comienza cuando un funcionario autorizado accede al formulario correspondiente y completa los datos requeridos, incluyendo el tipo de traslado, el elemento transportado, el origen, el destino, la prioridad y la fecha requerida.

El frontend envía la información al endpoint de creación mediante una petición HTTP POST.

El backend verifica la autenticación y autorización del funcionario, valida los campos recibidos y delega la operación al servicio TransferCreateService.

Este servicio comprueba las reglas de negocio correspondientes, incluyendo la validez de los tipos seleccionados y las condiciones establecidas para el recorrido.

Una vez completadas las validaciones, el sistema registra la solicitud en MariaDB y la asocia al funcionario responsable de su creación.

Finalmente, la API devuelve una respuesta satisfactoria con el identificador de la solicitud, su código y la versión inicial del registro.

Si alguno de los datos recibidos es inválido o no cumple las condiciones requeridas, el sistema rechaza la operación e informa el motivo correspondiente.

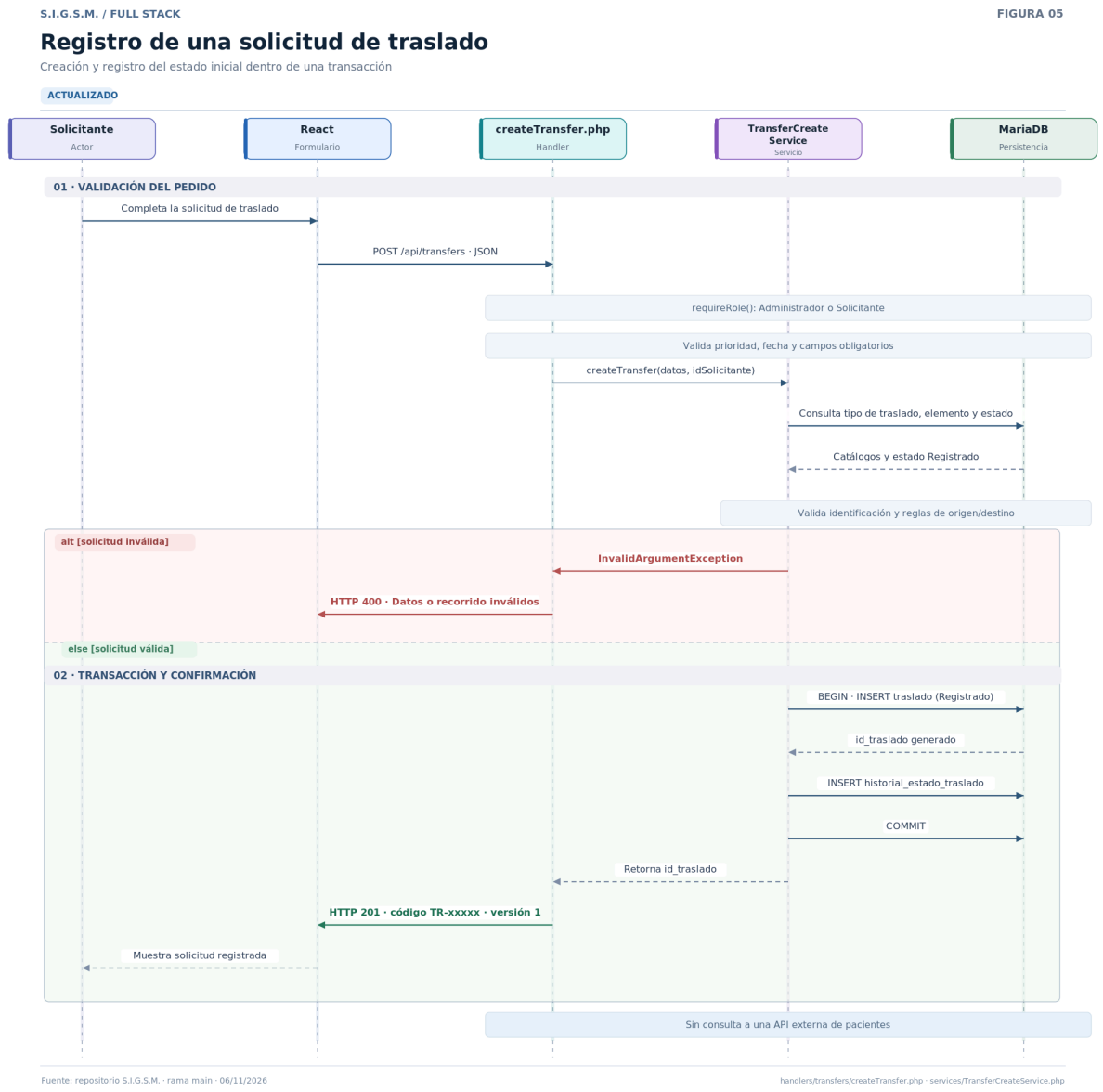


Figura 5. Diagrama de secuencia del registro de una solicitud de traslado.

5.4.2. Actualización del estado de un traslado

El siguiente diagrama representa el procedimiento mediante el cual un gestor de traslados realiza el avance de una solicitud al siguiente estado permitido.

El proceso comienza cuando el funcionario accede al seguimiento de un traslado y solicita la actualización de su estado.

El frontend envía una petición a la API, incluyendo el identificador del traslado y la versión del registro utilizada durante la operación.

El backend comprueba los permisos del funcionario y delega el procedimiento al servicio TransferStateService.

Este servicio consulta la información actual del traslado y verifica que la transición solicitada sea válida, que se cumplan las condiciones necesarias para realizarla y que la versión recibida coincida con la registrada en la base de datos.

Cuando todas las validaciones resultan satisfactorias, el sistema actualiza el estado del traslado y registra el cambio en la tabla historial_estado_traslado, conservando la información necesaria para su trazabilidad.

Posteriormente, la API devuelve el resultado de la operación y el frontend actualiza la información mostrada al funcionario.

El diagrama también contempla situaciones alternativas, como transiciones no permitidas, incumplimiento de condiciones o conflictos de concurrencia. En estos casos, la actualización es rechazada y se conserva la consistencia de la información almacenada.

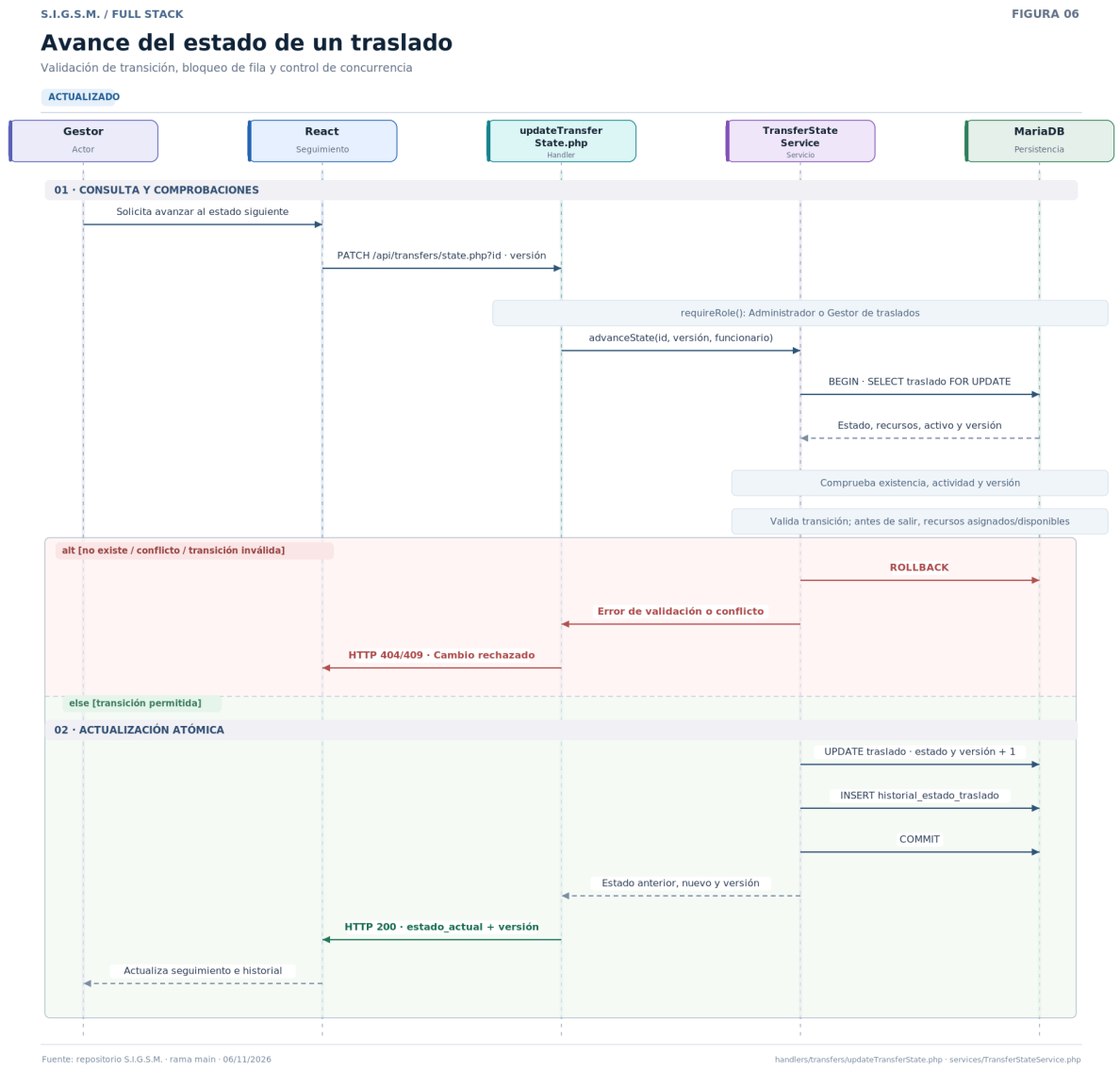


Figura 6. Diagrama de secuencia del proceso de actualización del estado de un traslado.

5.5. Diagrama de gestión de encuestas

El módulo de encuestas fue incorporado durante el desarrollo del proyecto y permite administrar formularios, configurar preguntas y almacenar las respuestas proporcionadas por los usuarios.

Para representar su funcionamiento se seleccionaron dos procedimientos: la creación de una encuesta y el registro de sus respuestas.

5.5.1. Creación de una encuesta

El siguiente diagrama representa el proceso mediante el cual un funcionario autorizado registra una nueva encuesta en S.I.G.S.M.

El proceso comienza cuando el gestor de documentación accede a la sección correspondiente y completa los datos generales del formulario, incluyendo su título, descripción, categoría opcional y configuración de anonimato.

Una vez enviada la información, el frontend realiza una petición HTTP POST al endpoint de encuestas.

El backend comprueba que el funcionario tenga permisos para realizar la operación y valida los datos recibidos.

Posteriormente, el handler correspondiente obtiene una conexión con MariaDB y utiliza el servicio SurveyCreateService para registrar la nueva encuesta.

Una vez finalizada la operación, la API devuelve una respuesta satisfactoria con el identificador generado, permitiendo continuar con la configuración de las preguntas del formulario.

En caso de recibir datos inválidos o producirse un error durante el registro, el sistema devuelve una respuesta de error y evita completar la operación.

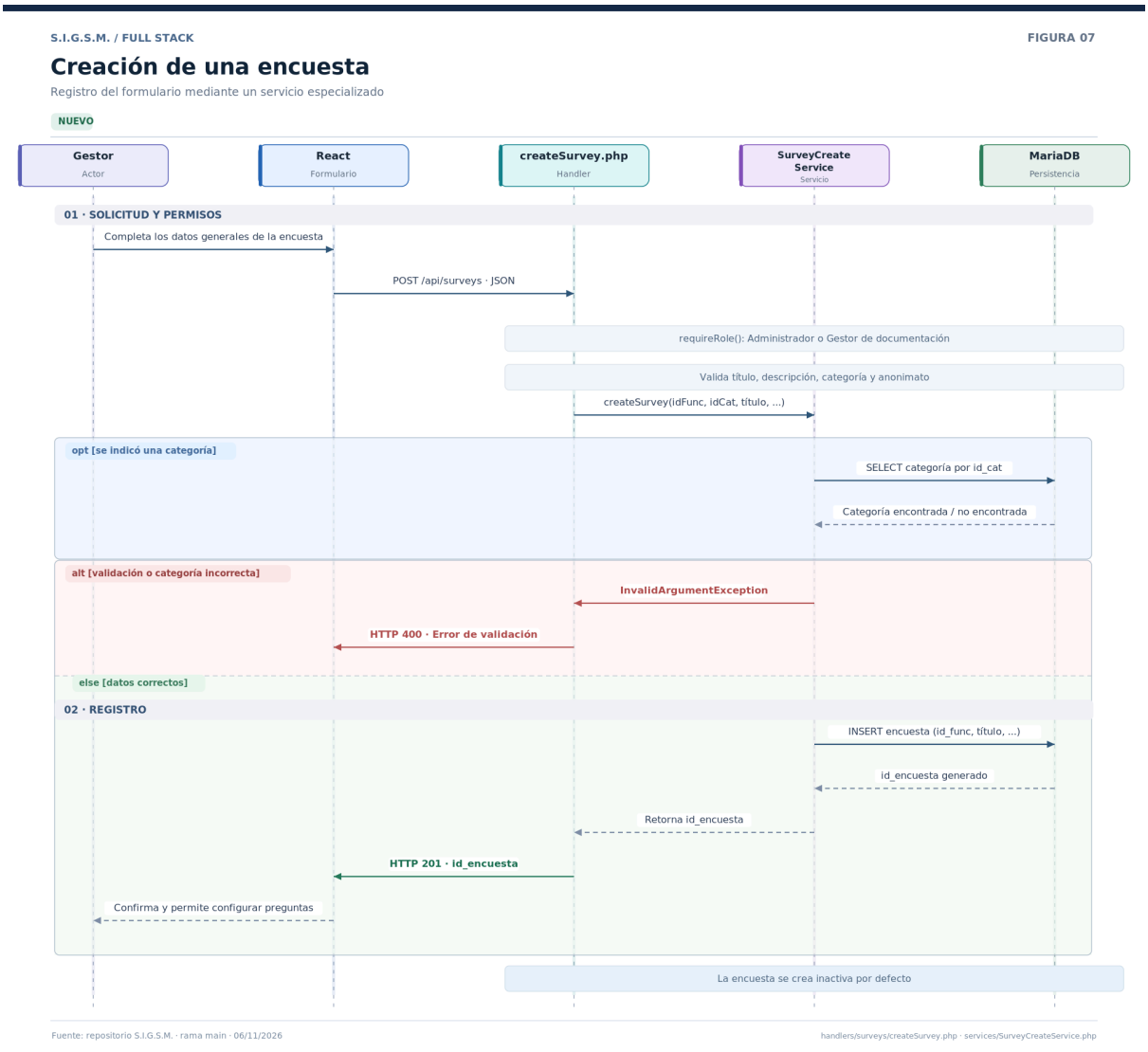


Figura 7. Diagrama de secuencia del proceso de creación de una encuesta.

5.5.2. Registro de respuestas de una encuesta

El siguiente diagrama representa el procedimiento mediante el cual un usuario responde una encuesta publicada.

El proceso comienza cuando el usuario accede al formulario y completa las preguntas correspondientes.

Una vez enviadas las respuestas, el frontend realiza una petición a la API con el identificador de la encuesta y la información proporcionada.

El backend procesa los datos recibidos, comprueba su estructura y valida que las respuestas correspondan a los formatos admitidos.

Posteriormente, el servicio SurveyResponseCreateService realiza las comprobaciones necesarias sobre la encuesta y sus preguntas, incluyendo las condiciones de identificación, obligatoriedad y validez de las opciones seleccionadas.

Cuando las respuestas cumplen las condiciones establecidas, se registra la información en las entidades respuesta_encuesta y detalle_respuesta.

Finalmente, la API devuelve una confirmación y el frontend informa al usuario que el formulario fue enviado correctamente.

Si la encuesta no se encuentra disponible, existen respuestas inválidas o se incumple alguna de las reglas definidas, el sistema rechaza el registro y devuelve el error correspondiente.

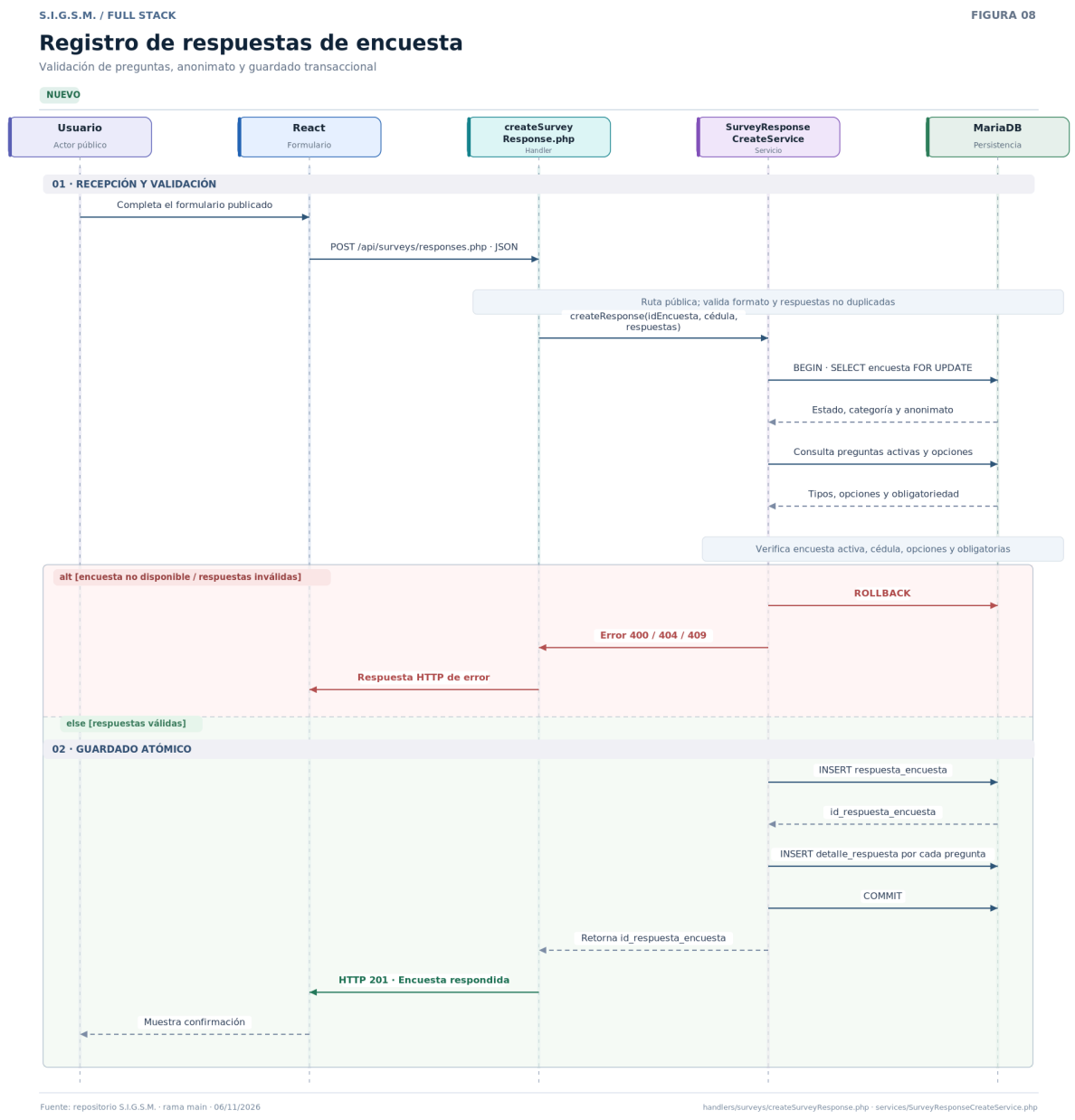


Figura 8. Diagrama de secuencia del registro de respuestas de una encuesta.

Los diagramas presentados permiten visualizar la interacción entre los principales componentes de S.I.G.S.M. y complementan la descripción de su arquitectura.

Asimismo, muestran cómo las validaciones y reglas de negocio se ejecutan desde el backend, manteniendo separadas las responsabilidades de presentación, procesamiento y persistencia de información.

6. Testing funcional

6.1. Introducción y objetivos de las pruebas

El proceso de testing funcional de S.I.G.S.M. tuvo como objetivo comprobar que las funcionalidades implementadas responden correctamente a los requerimientos establecidos durante el desarrollo del proyecto.

Las pruebas se centraron en los principales módulos de la aplicación, incluyendo autenticación y administración de funcionarios, gestión documental, encuestas y gestión y seguimiento de traslados.

Para ello, se definieron casos de prueba que contemplan tanto situaciones de funcionamiento esperado como escenarios alternativos, validaciones y condiciones de error.

El proceso de testing permitió evaluar el comportamiento de la aplicación desde la perspectiva del usuario y comprobar la integración entre el frontend, la API REST y la base de datos.

Los principales objetivos fueron verificar el cumplimiento de los requerimientos funcionales, comprobar las reglas de negocio implementadas, identificar posibles errores y registrar los resultados obtenidos.

6.2. Metodología y entorno de pruebas

La metodología utilizada se basó principalmente en pruebas funcionales de caja negra, complementadas con pruebas de integración, validación, autorización y exploración de la interfaz.

Las comprobaciones se realizaron progresivamente durante el desarrollo, permitiendo identificar errores y corregirlos antes de continuar con la implementación de nuevas funcionalidades.

El entorno utilizado estuvo compuesto por las siguientes tecnologías y herramientas:

- **Frontend**: React, Vite y Material UI, ejecutados desde un navegador web.
- **Backend**: API REST desarrollada en PHP 8.2.
- **Base de datos**: MariaDB.
- **Servidor**: Fedora Server ejecutado en una máquina virtual.
- **XAMPP**: utilizado para ejecutar los servicios del backend y la base de datos durante el desarrollo.
- **Herramientas de terminal**: utilizadas para realizar solicitudes HTTP y comprobar las respuestas de la API.
- **Google Sheets**: utilizado para registrar y organizar los casos de prueba.

Para documentar las comprobaciones se elaboró una planilla compartida que contiene 36 casos de prueba.

Cada caso incluye su identificador, funcionalidad evaluada, precondiciones, pasos de ejecución, resultado esperado, tipo de prueba, prioridad, estado, resultado obtenido, responsable y fecha de ejecución.

El alcance de las pruebas se limitó a las funcionalidades implementadas por el equipo. No se realizaron pruebas de integración con los sistemas internos del Hospital de Clínicas, debido a que no se proporcionaron los accesos ni la documentación necesarios.

6.3. Pruebas de autenticación y autorización

Las pruebas de autenticación se orientaron a comprobar los mecanismos de acceso al sistema y las restricciones establecidas según el estado y los roles de los funcionarios.

Se verificaron las operaciones de inicio y cierre de sesión, la validación de credenciales y la protección de funcionalidades privadas.

Entre los escenarios evaluados se encuentran el inicio de sesión con credenciales correctas, el rechazo de contraseñas incorrectas, el acceso de funcionarios inactivos y las solicitudes realizadas sin una sesión válida.

También se comprobaron las restricciones de autorización según los roles asignados, las consultas de funcionarios y el registro de solicitudes de acceso.

En total, se ejecutaron 8 casos de prueba, identificados mediante los códigos AUT-01 a AUT-05 y USR-01 a USR-03.

Los resultados obtenidos fueron satisfactorios, comprobándose que las operaciones evaluadas respondieron de acuerdo con las condiciones de autenticación y autorización establecidas.

6.4. Pruebas del módulo de documentación

Las pruebas del módulo de documentación tuvieron como objetivo verificar las operaciones de administración de categorías y documentos, así como las validaciones relacionadas con los archivos y las restricciones de acceso.

Se comprobaron funcionalidades como la creación de categorías, el rechazo de datos incompletos, la carga de documentos PDF, la modificación de su información, la desactivación y la eliminación de documentos.

Asimismo, se verificó el acceso a los documentos publicados y las restricciones correspondientes a aquellos que se encuentran inactivos.

Estas pruebas permitieron comprobar tanto las operaciones realizadas desde la interfaz como su correspondiente procesamiento desde la API y el almacenamiento de la información.

Se ejecutaron 8 casos de prueba, identificados con los códigos DOC-01 a DOC-08.

Los resultados coincidieron con los comportamientos esperados, verificándose las funcionalidades principales de administración documental contempladas en el plan.

6.5. Pruebas del módulo de encuestas

Las pruebas del módulo de encuestas se centraron en verificar las operaciones de creación, publicación, consulta, respuesta y visualización de resultados.

Se comprobó el registro de nuevas encuestas, la aplicación de las validaciones necesarias para su publicación y el comportamiento de los formularios ante diferentes condiciones de respuesta.

También se evaluaron situaciones relacionadas con respuestas completas e incompletas, así como la consulta de encuestas publicadas y la consistencia de los resultados obtenidos a partir de la información almacenada.

Estas comprobaciones permitieron verificar el funcionamiento general del módulo y las principales reglas de negocio asociadas a la administración y utilización de los formularios.

En total, se ejecutaron 6 casos de prueba, identificados con los códigos ENC-01 a ENC-06.

Los resultados obtenidos fueron satisfactorios para todos los escenarios evaluados.

6.6. Pruebas del módulo de traslados

El módulo de traslados recibió especial atención durante el testing debido a la cantidad de reglas de negocio involucradas en la creación, asignación de recursos y seguimiento de las solicitudes.

Las pruebas contemplaron el registro de nuevos traslados, la validación de fechas y recorridos, la consulta de recursos disponibles y la asignación de vehículos y funcionarios.

También se verificó que el sistema rechazara asignaciones incompatibles o aquellas que presentaran conflictos de disponibilidad.

En cuanto al seguimiento, se comprobaron las transiciones válidas entre los diferentes estados, el rechazo de cambios fuera de secuencia y la detección de conflictos cuando se intenta modificar un traslado utilizando una versión desactualizada del registro.

Finalmente, se evaluaron las operaciones de cancelación de solicitudes y consulta del historial de traslados.

En total, se ejecutaron 11 casos de prueba, identificados con los códigos TRA-01 a TRA-11.

Los resultados fueron satisfactorios, comprobándose el cumplimiento de las principales reglas de negocio contempladas en el plan de testing.

6.7. Pruebas de validación y manejo de errores

Además de verificar el funcionamiento habitual de los módulos, se realizaron pruebas destinadas a comprobar la respuesta del sistema ante entradas incorrectas, operaciones no autorizadas y situaciones que incumplen las reglas de negocio.

Estas comprobaciones permitieron verificar que las operaciones inválidas fueran rechazadas y que la aplicación devolviera respuestas adecuadas sin comprometer la consistencia de los datos.

Para complementar los casos de prueba se utilizaron tablas de decisión, que permiten representar el comportamiento esperado ante diferentes combinaciones de condiciones.

6.7.1. Control de acceso a funcionalidades restringidas

La primera tabla representa las condiciones necesarias para acceder a una funcionalidad protegida.

Sesión válida	Rol autorizado	Resultado
Sí	Sí	Acceso permitido
Sí	No	Acceso denegado
No	Sí	Acceso denegado
No	No	Acceso denegado

El acceso únicamente debe permitirse cuando el funcionario dispone de una sesión válida y cuenta con el rol requerido.

Esto permite comprobar que la autenticación no sustituye al control de autorización y que ambas condiciones deben cumplirse para ejecutar determinadas operaciones.

6.7.2. Asignación de recursos a un traslado

La segunda tabla contempla algunas de las condiciones principales que intervienen durante la asignación de vehículos y funcionarios.

Recursos disponibles	Recursos compatibles	Versión válida	Resultado
Sí	Sí	Sí	Asignación permitida
No	Sí	Sí	Asignación rechazada
Sí	No	Sí	Asignación rechazada

Sí	Sí	No	Conflicto de versión
No	No	Sí	Asignación rechazada

La operación debe cumplir las condiciones de disponibilidad y compatibilidad de recursos, así como las comprobaciones correspondientes al control de concurrencia.

Además de estas condiciones, el sistema realiza otras validaciones relacionadas con el estado del traslado y los permisos del funcionario que ejecuta la operación.

6.7.3. Actualización de estados de traslado

La tercera tabla representa las condiciones generales necesarias para avanzar una solicitud al siguiente estado.

Transición válida	Condiciones requeridas cumplidas	Versión válida	Resultado
Sí	Sí	Sí	Estado actualizado
No	Sí	Sí	Cambio rechazado
Sí	No	Sí	Cambio rechazado
Sí	Sí	No	Conflicto de versión

La actualización únicamente debe completarse cuando la transición corresponde al flujo establecido, se cumplen las condiciones necesarias y la versión del registro es válida.

Esto permite evitar cambios de estado incorrectos y modificaciones que podrían sobrescribir información actualizada previamente por otro funcionario.

Las tablas presentadas resumen las principales reglas de decisión consideradas durante el testing y complementan los casos de prueba registrados en la planilla correspondiente.

6.8. Resultados generales de las pruebas

Los resultados del testing se registraron en una planilla compartida de Google Sheets, permitiendo mantener un historial organizado de los casos ejecutados.

La ejecución documentada estuvo a cargo de Martín Cabrera, con fecha 5 de noviembre de 2026.

A continuación, se presenta un resumen de los resultados obtenidos:

Módulo	Casos ejecutados	Resultados satisfactorios
Autenticación	5	5
Administración de funcionarios	3	3
Gestión documental	8	8
Encuestas	6	6
Traslados	11	11
Interfaz de usuario	3	3
Total	**36**	**36**

Las tres pruebas exploratorias de interfaz, identificadas como UI-01 a UI-03, estuvieron orientadas a comprobar la navegación, la adaptación a diferentes tamaños de pantalla y la presentación de mensajes de error.

De acuerdo con el registro realizado, los 36 casos de prueba fueron ejecutados satisfactoriamente, alcanzándose un cumplimiento del 100 % de los resultados esperados para los escenarios evaluados.

No se registraron incidencias durante esta ejecución del plan de testing.

Es importante destacar que estos resultados corresponden únicamente a los casos definidos y ejecutados, por lo que no implican necesariamente la ausencia de errores en funcionalidades o escenarios que no hayan sido contemplados.

Las pruebas permitieron verificar operaciones habituales, situaciones alternativas y condiciones de error, especialmente en los módulos que presentan mayores restricciones de acceso y reglas de negocio.

En particular, las comprobaciones del módulo de traslados permitieron evaluar aspectos relevantes como la disponibilidad de recursos, la prevención de asignaciones incompatibles, las transiciones entre estados y el control de versiones de los registros.

Asimismo, la utilización de una planilla compartida permitió centralizar la información, facilitar su consulta y conservar evidencia documental de los resultados obtenidos.

En conclusión, el testing funcional proporcionó resultados favorables sobre el funcionamiento de las principales funcionalidades implementadas en S.I.G.S.M. y

permitió comprobar su comportamiento dentro del entorno de desarrollo y pruebas utilizado.

No obstante, será necesario continuar realizando comprobaciones ante futuras modificaciones, así como incorporar pruebas de aceptación con usuarios de la institución antes de una eventual implementación en producción.

Plan de testing de S.I.G.S.M.: Plan testing - Hojas de cálculo de Google

7. Conclusiones

7.1. Resultados del desarrollo

La tercera entrega de Programación Full Stack permitió consolidar la documentación técnica de S.I.G.S.M. y presentar una visión general de la solución desarrollada, abarcando el modelo de datos definitivo, la arquitectura del sistema, los diagramas UML y las pruebas funcionales realizadas.

A lo largo del proyecto se logró desarrollar una aplicación web que integra diferentes funcionalidades administrativas mediante una arquitectura cliente-servidor, utilizando React para el frontend, PHP para el backend y MariaDB como sistema gestor de base de datos.

Uno de los principales resultados fue la evolución de la arquitectura hacia una estructura más modular, especialmente en el backend, donde se separaron las responsabilidades mediante endpoints, handlers y clases de servicio especializadas. Esta organización permitió incorporar nuevas funcionalidades y mantener una estructura de código más ordenada conforme aumentaba la complejidad del sistema.

Asimismo, el modelo de datos experimentó diferentes modificaciones para adaptarse a las necesidades identificadas durante el desarrollo. La incorporación de nuevas entidades, relaciones y restricciones permitió representar de manera más precisa las operaciones correspondientes a los módulos de documentación, encuestas y gestión de traslados.

Desde el punto de vista funcional, se implementaron mecanismos de autenticación y autorización, administración de funcionarios, gestión documental, creación y administración de encuestas, registro de solicitudes de traslado, asignación de recursos, seguimiento de estados y consulta del historial de operaciones.

También se incorporó una configuración de despliegue mediante Docker Compose, que permite ejecutar los diferentes componentes de la aplicación en contenedores independientes, facilitando su instalación y mantenimiento.

En cuanto a la validación del sistema, se ejecutaron 36 casos de prueba funcionales y exploratorios, obteniéndose resultados satisfactorios en todos los escenarios evaluados. Estas comprobaciones permitieron verificar las principales operaciones y reglas de negocio implementadas, proporcionando evidencia favorable sobre el funcionamiento de la aplicación en el entorno de pruebas.

En conjunto, los resultados obtenidos reflejan la evolución del proyecto desde las primeras etapas de diseño hasta una aplicación funcional, con una estructura técnica organizada y documentación que facilita su comprensión y posible mantenimiento.

7.2. Limitaciones conocidas

A pesar de los avances alcanzados, durante el desarrollo de S.I.G.S.M. surgieron determinadas limitaciones que condicionaron el alcance final de algunas funcionalidades.

Una de las principales dificultades estuvo relacionada con la integración con los sistemas internos del Hospital de Clínicas. Inicialmente se contemplaba la posibilidad de consultar información de pacientes mediante servicios proporcionados por la institución, pero no se dispuso de los accesos ni de la documentación técnica necesarios para implementar dicha comunicación.

Por este motivo, se decidió desarrollar las funcionalidades de manera independiente, utilizando los datos ingresados desde la aplicación y dejando pendiente la integración con los sistemas institucionales.

Otra limitación corresponde a la validación del sistema en un entorno real de utilización. Si bien se realizaron pruebas funcionales y demostraciones durante el desarrollo, todavía no se llevaron a cabo pruebas de aceptación formales con los funcionarios del Hospital de Clínicas que utilizarían la aplicación.

Asimismo, la configuración actual de despliegue está orientada a entornos de desarrollo y demostración. Su implementación definitiva sobre la infraestructura institucional requerirá realizar las adaptaciones correspondientes en materia de conectividad, seguridad, configuración del servidor y administración de los servicios.

Por último, aunque los resultados del testing fueron satisfactorios, las pruebas realizadas se limitaron a los escenarios definidos en el plan. Por lo tanto, no es

posible garantizar la ausencia de errores en situaciones que no hayan sido contempladas.

Estas limitaciones no impiden valorar los resultados alcanzados, pero deberán considerarse antes de una eventual puesta en producción del sistema.

7.3. Posibles mejoras futuras

La arquitectura modular adoptada durante el desarrollo proporciona una base que permite incorporar nuevas funcionalidades y realizar modificaciones sin necesidad de reorganizar completamente la aplicación.

Entre las principales mejoras identificadas se encuentra la integración con los sistemas internos del Hospital de Clínicas, especialmente para consultar información de pacientes y reducir la necesidad de introducir determinados datos manualmente.

También se considera conveniente continuar mejorando las validaciones y reglas de negocio del módulo de traslados. En particular, se identificó como posible mejora solicitar obligatoriamente una fecha y hora de ejecución cuando se registre una solicitud con prioridad urgente, facilitando la planificación y coordinación de los recursos disponibles.

En cuanto a la calidad del software, sería beneficioso ampliar la cobertura de las pruebas automatizadas, incorporando nuevos casos de prueba sobre los servicios del backend y las interacciones entre los diferentes componentes del sistema.

Asimismo, se recomienda realizar pruebas de aceptación con funcionarios de la institución para evaluar la facilidad de uso de las interfaces, comprobar su adecuación a los procedimientos reales e identificar posibles ajustes.

Desde el punto de vista de la infraestructura, será necesario reforzar la configuración de seguridad antes de una implementación definitiva, incorporando mecanismos adecuados para proteger las comunicaciones, controlar los accesos y garantizar la disponibilidad de la información.

Finalmente, se considera importante mantener actualizada la documentación técnica conforme se incorporen modificaciones, especialmente en lo relacionado con el modelo de datos, los endpoints de la API y los procedimientos de instalación y mantenimiento.

En conclusión, la tercera entrega de Full Stack permitió documentar y evaluar la solución desarrollada para S.I.G.S.M., demostrando la integración de los conocimientos adquiridos durante la formación en desarrollo web, bases de datos y arquitectura de software.

Si bien existen aspectos pendientes de implementación y validación institucional, se considera que el proyecto alcanzó sus principales objetivos técnicos dentro del entorno académico, proporcionando una aplicación funcional y una base organizada para su mantenimiento y futuras ampliaciones.

8. Bibliografía y referencias

Para la elaboración del presente documento se utilizaron principalmente el código fuente de S.I.G.S.M., la documentación técnica desarrollada durante las entregas anteriores y los registros de pruebas funcionales del proyecto.

A continuación, se presentan las principales fuentes consultadas.

Empre S.A.S. (2026). S.I.G.S.M. — Repositorio oficial del proyecto. GitHub. https://github.com/MrGalletah/S.I.G.S.M---Proyecto-UTU

Empre S.A.S. (2026). S.I.G.S.M. — Documentación técnica del frontend. GitHub. https://github.com/MrGalletah/S.I.G.S.M---Proyecto-UTU/tree/main/prototipos/frontend

Empre S.A.S. (2026). S.I.G.S.M. — Documentación técnica del backend y API REST. GitHub. https://github.com/MrGalletah/S.I.G.S.M---Proyecto-UTU/tree/main/prototipos/backend

Empre S.A.S. (2026). S.I.G.S.M. — Configuración y documentación del despliegue mediante Docker. GitHub. https://github.com/MrGalletah/S.I.G.S.M---Proyecto-UTU/tree/main/prototipos/docker

Empre S.A.S. (2026). S.I.G.S.M. — Ingeniería de Software, tercera entrega. Instituto Superior Brazo Oriental.
Documento interno del proyecto. Incluye la revisión de diagramas de clases y el plan de testing funcional.

Empre S.A.S. (2026). S.I.G.S.M. — Plan de testing funcional. Google Sheets.
Planilla de pruebas que contiene los 36 casos ejecutados, sus resultados, responsables y fechas de ejecución. Plan testing - Hojas de cálculo de Google

Empre S.A.S. (2026). S.I.G.S.M. — Documentación de Full Stack, primera y segunda entrega. Instituto Superior Brazo Oriental.

Documentación previa utilizada como referencia para analizar la evolución del modelo de datos, la arquitectura y las funcionalidades implementadas.

9. Anexos

En esta sección se presentan los recursos y documentos complementarios utilizados durante el desarrollo de S.I.G.S.M., con el objetivo de facilitar la consulta del código fuente, la estructura de la base de datos y la documentación técnica del proyecto.

Estos materiales permiten ampliar la información presentada en los apartados anteriores y sirven como referencia para futuras tareas de mantenimiento, modificación o ampliación del sistema.

9.1. Repositorio GitHub del proyecto

El código fuente de S.I.G.S.M. se encuentra disponible en un repositorio público de GitHub, utilizado durante el desarrollo para centralizar los archivos del proyecto y mantener un historial de las modificaciones realizadas.

El repositorio contiene el código del frontend, la API REST, los scripts de la base de datos, la configuración de Docker y la documentación correspondiente a las diferentes entregas académicas.

Repositorio oficial de S.I.G.S.M.:
https://github.com/MrGalletah/S.I.G.S.M---Proyecto-UTU

La rama main contiene la versión principal del proyecto y constituye la referencia utilizada para elaborar la documentación técnica de esta entrega.

9.2. Scripts SQL definitivos

La estructura de la base de datos de S.I.G.S.M. se encuentra definida mediante cuatro scripts SQL, organizados según los diferentes módulos funcionales de la aplicación.

Estos archivos incluyen la creación de las tablas, claves primarias, claves foráneas, restricciones de integridad e índices utilizados por el sistema.

Los scripts se encuentran disponibles en el siguiente directorio del repositorio: https://github.com/MrGalletah/S.I.G.S.M---Proyecto-UTU/tree/main/prototipos/backend/database

Los archivos que componen el modelo definitivo son:

- **dbSchema_01.sql**: funcionarios, roles y asignación de permisos.
- **dbSchema_02.sql**: categorías y documentos.
- **dbSchema_03.sql**: vehículos, traslados, recursos y seguimiento de estados.
- **dbSchema_04.sql**: encuestas, preguntas, opciones y respuestas.

Estos scripts corresponden al modelo de datos documentado en el apartado 2 y permiten reproducir la estructura de la base de datos en una instalación nueva.

9.3. Documentación técnica complementaria

Durante el desarrollo de S.I.G.S.M. se elaboraron diferentes documentos técnicos que complementan la información presentada en esta entrega.

Entre ellos se encuentran las entregas de Ingeniería de Software, que incluyen el análisis de requerimientos, los diagramas UML, la evolución de la estructura interna del sistema y el plan de testing funcional.

Asimismo, se elaboró un manual de instalación y mantenimiento que describe los requisitos necesarios para ejecutar la aplicación, la configuración mediante Docker Compose, la administración de los servicios y los procedimientos de respaldo y recuperación de información.

La documentación del proyecto también se complementa con los archivos README disponibles en los directorios del frontend, backend y Docker, que contienen información específica sobre la estructura, configuración y funcionamiento de cada componente.

Estos recursos constituyen material de apoyo para comprender las decisiones técnicas adoptadas con el afán de facilitar futuras modificaciones o tareas de mantenimiento. Los recursos mencionados se encuentran disponibles en el repositorio de GitHub.